Factory Hub

Ecossistema Integrado de Software Industrial (Indústria 4.0)

Etapa 1: Entendendo a solução

O Factory Hub simula uma fábrica inteligente, na qual máquinas industriais enviam informações sobre seu funcionamento para uma plataforma na nuvem.

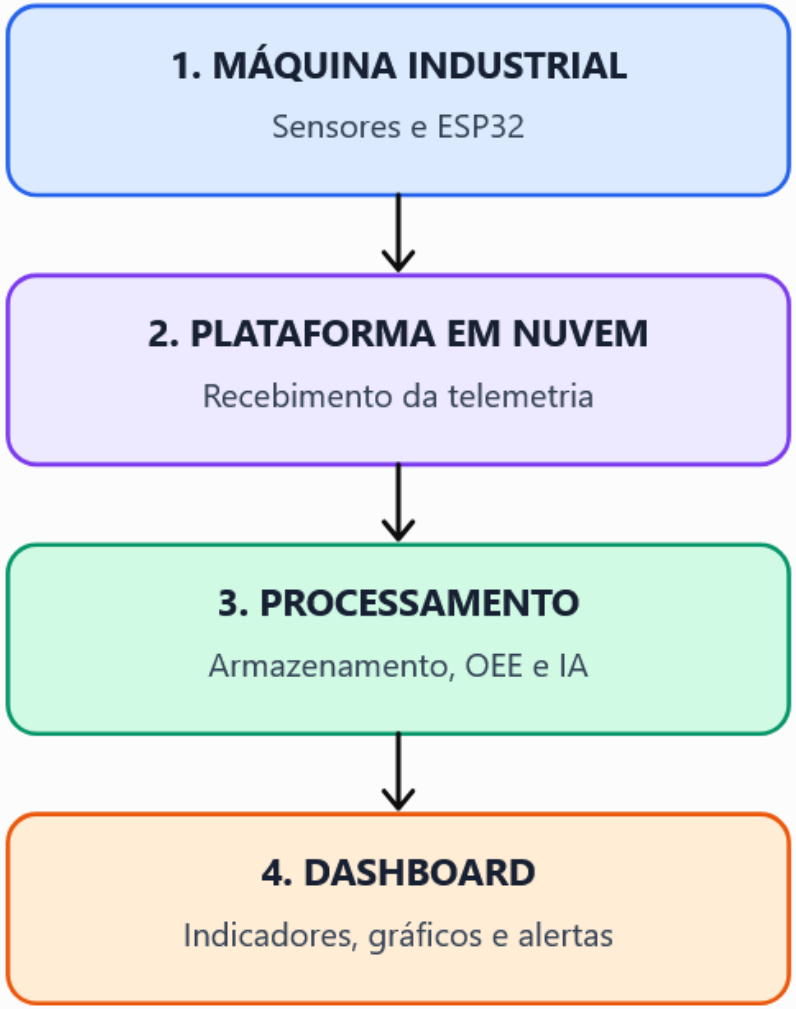
1. Entendendo a solução

- Essa plataforma possui quatro objetivos principais:
  - Monitoramento: acompanhar temperatura, vibração, rotação e produção das máquinas.
  - Armazenamento: preservar o histórico das medições e os dados operacionais.
  - Inteligência: utilizar Machine Learning para identificar riscos de anomalias.
  - Gestão: apresentar indicadores de eficiência, como OEE, e alertas aos usuários.
- O projeto integra quatro disciplinas: Automação Industrial, Ciência de Dados, Computação em Nuvem e DevOps.

2. Primeiro desenho — Visão conceitual

Antes de detalhar AWS, bancos de dados ou containers, vamos representar apenas os grandes blocos.

Esse desenho ainda não representa a arquitetura técnica completa. Ele nos ajuda a compreender o problema antes de introduzir as tecnologias.



3. Primeira camada — Máquina industrial e ESP32

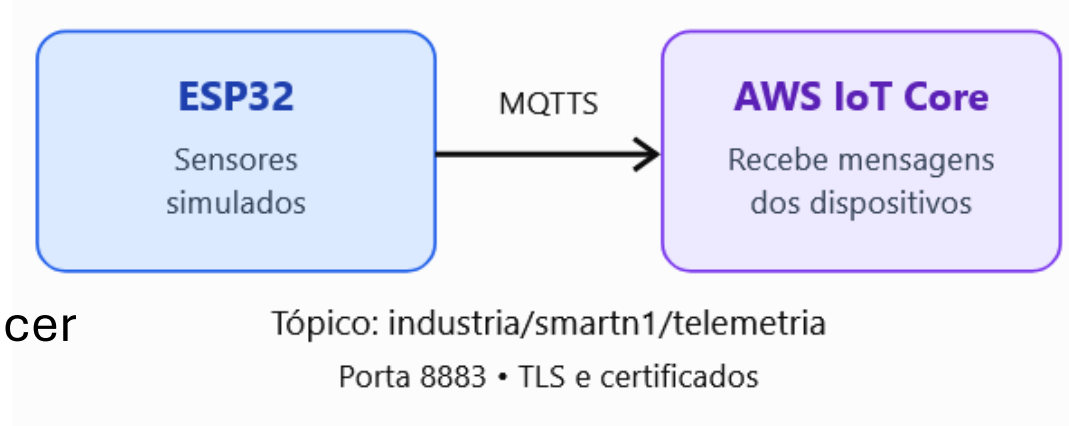
- Qual é o papel do ESP32?
  - O ESP32 representa um equipamento instalado próximo à máquina industrial, responsável por obter ou simular as medições e enviá-las à plataforma.

Informação	Exemplo	Significado
device_id	esp32-torno-01	Identificação da máquina
temperatura	72,8	Temperatura medida
vibracao	2,45	Vibração medida
rpm	3480	Velocidade de rotação
status	OPERATIONAL	Estado operacional
pecas_produzidas	15	Quantidade produzida
pecas_defeito	1	Quantidade defeituosa

3. Primeira camada — Máquina industrial e ESP32

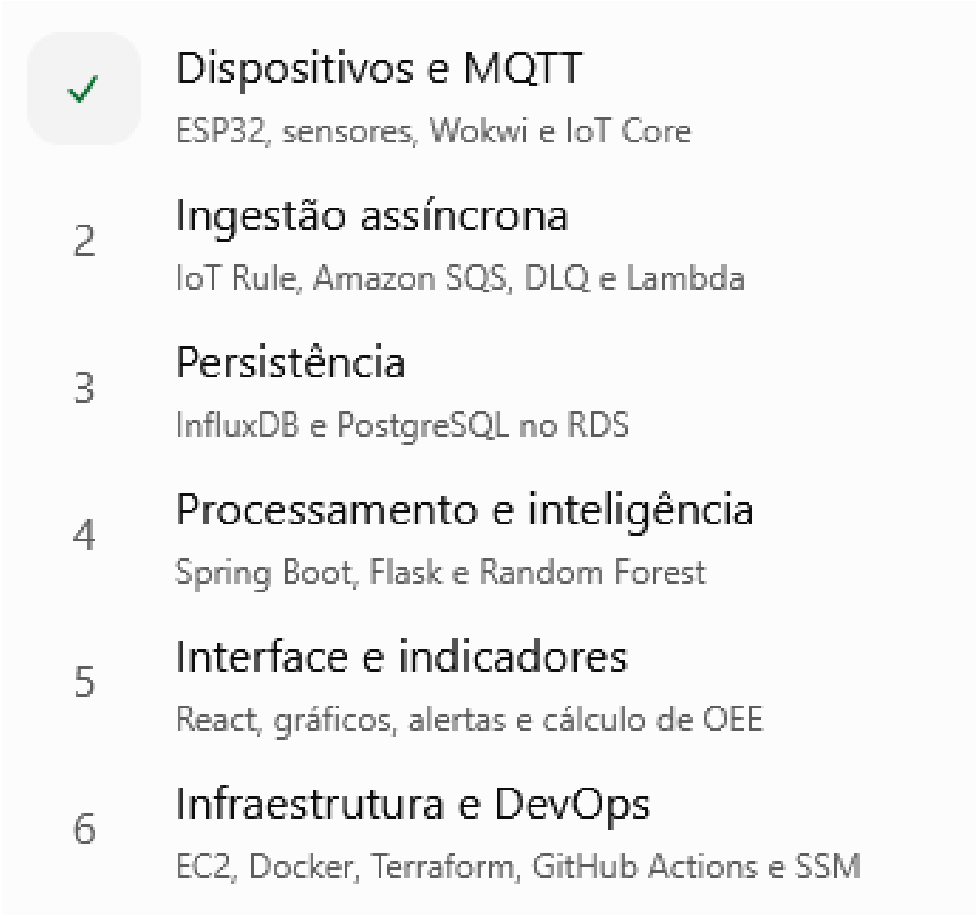
- O dispositivo publica esses dados utilizando MQTT, um protocolo de comunicação leve, bastante utilizado em IoT.
- No projeto, o destino é o tópico:
- industria/smartn1/telemetria
- O ESP32 se conecta ao AWS IoT Core por MQTT seguro, utilizando certificados digitais.

Por que usar o AWS IoT Core em vez de enviar diretamente ao backend Java?
Porque ele oferece uma camada especializada no recebimento e autenticação de mensagens dos dispositivos. Assim, a máquina não precisa conhecer a API Java nem os bancos de dados.
Isso introduz um princípio importante de arquitetura: desacoplamento entre dispositivos e aplicações.



4. O que virá nas próximas etapas

- Vamos evoluir o mesmo desenho, sem introduzir todos os serviços de uma só vez.



Etapa 2: Ingestão de dados na AWS

Nesta etapa, vamos entender como a telemetria recebida pelo AWS IoT Core é encaminhada, armazenada temporariamente e processada antes de chegar ao InfluxDB.

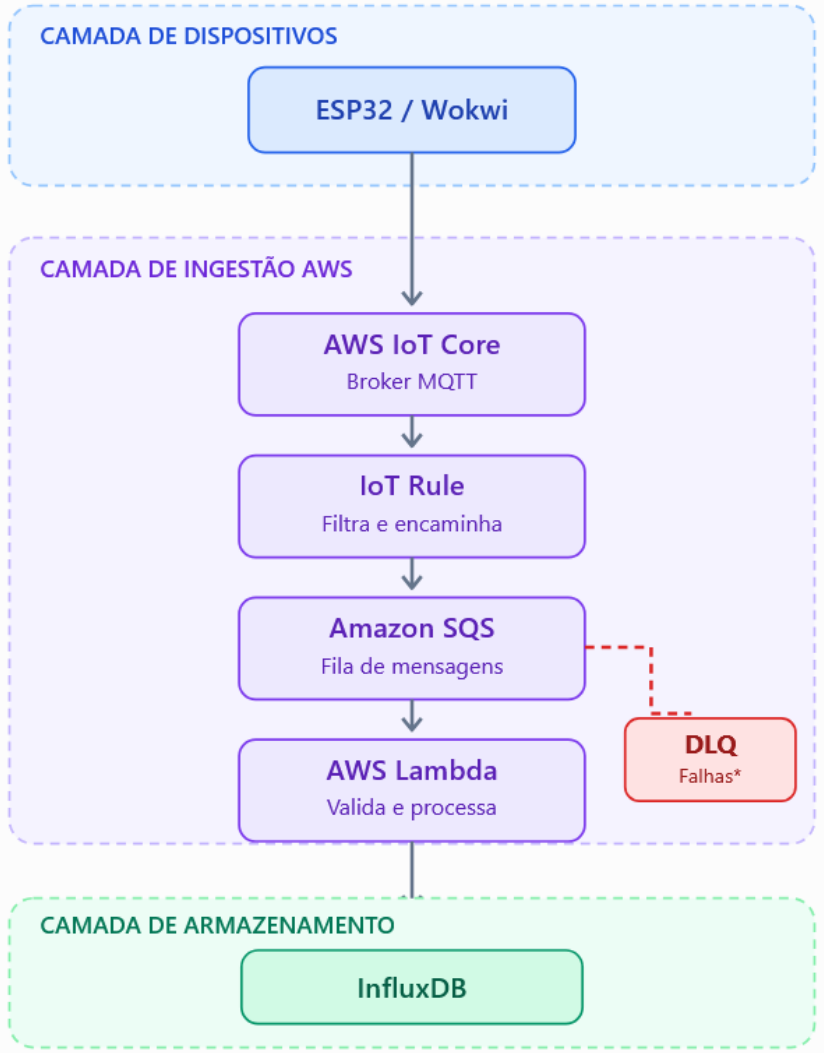
A arquitetura passa a incorporar quatro componentes: IoT Rule, Amazon SQS, Dead Letter Queue (DLQ) e AWS Lambda.

1. Ampliando nossa arquitetura

O fluxo principal é:

ESP32 → IoT Core → IoT Rule → SQS → Lambda → InfluxDB

Observe que a Lambda não recebe diretamente as mensagens do ESP32. Existe uma fila entre o recebimento e o processamento.
Essa decisão é fundamental para a confiabilidade do sistema.



2. AWS IoT Rule — Direcionamento das mensagens

- Na etapa anterior, o ESP32 publicou uma mensagem MQTT no tópico:

industria/smartn1/telemetria

- O AWS IoT Core recebe essa publicação. Mas como decidir o que fazer com ela?
- A resposta é a IoT Rule.
- Exemplo didático de regra SQL

```
SELECT *
FROM 'industria/smartn1/telemetria'
```

2. AWS IoT Rule — Direcionamento das mensagens

- Essa regra seleciona as mensagens publicadas no tópico especificado. Sua ação configurada pode encaminhá-las para uma fila SQS.
- O processamento ocorre assim:
  - O ESP32 publica o JSON no tópico MQTT.
  - O IoT Core recebe a mensagem e verifica as regras aplicáveis.
  - A IoT Rule seleciona o conteúdo da mensagem.
  - A ação da regra envia o conteúdo para o Amazon SQS.

2. AWS IoT Rule — Direcionamento das mensagens

- Por que não enviar diretamente para a Lambda?
  - Tecnicamente, o IoT Core pode acionar uma Lambda diretamente. Porém, o projeto escolhe uma fila intermediária para desacoplar o recebimento das mensagens do processamento.
  - Isso permite absorver picos de telemetria e lidar melhor com indisponibilidades temporárias.

3. Amazon SQS — A fila de mensagens

- Imagine que temos 100 máquinas publicando informações simultaneamente.
- Se todas enviarem mensagens diretamente ao processador, ele precisará lidar com a carga no mesmo momento em que elas chegam.
- O SQS resolve parte desse problema ao funcionar como um buffer de mensagens.



3. Amazon SQS — A fila de mensagens

- O que acontece quando a Lambda está indisponível?
  - As mensagens podem permanecer na fila, respeitando o período de retenção configurado.
  - Quando o processamento estiver disponível novamente, a integração entre SQS e Lambda poderá continuar consumindo as mensagens.
  - Isso aumenta a resiliência do Factory Hub.

3. Amazon SQS — A fila de mensagens

- Propriedades importantes do SQS

Propriedade	Finalidade
Retenção	Define quanto tempo uma mensagem pode permanecer na fila
Visibility Timeout	Evita que uma mensagem em processamento seja entregue imediatamente a outro consumidor
Long Polling	Permite aguardar mensagens, reduzindo consultas vazias
Redrive Policy	Define quando uma mensagem deve ser movida para a DLQ

4. DLQ — Dead Letter Queue

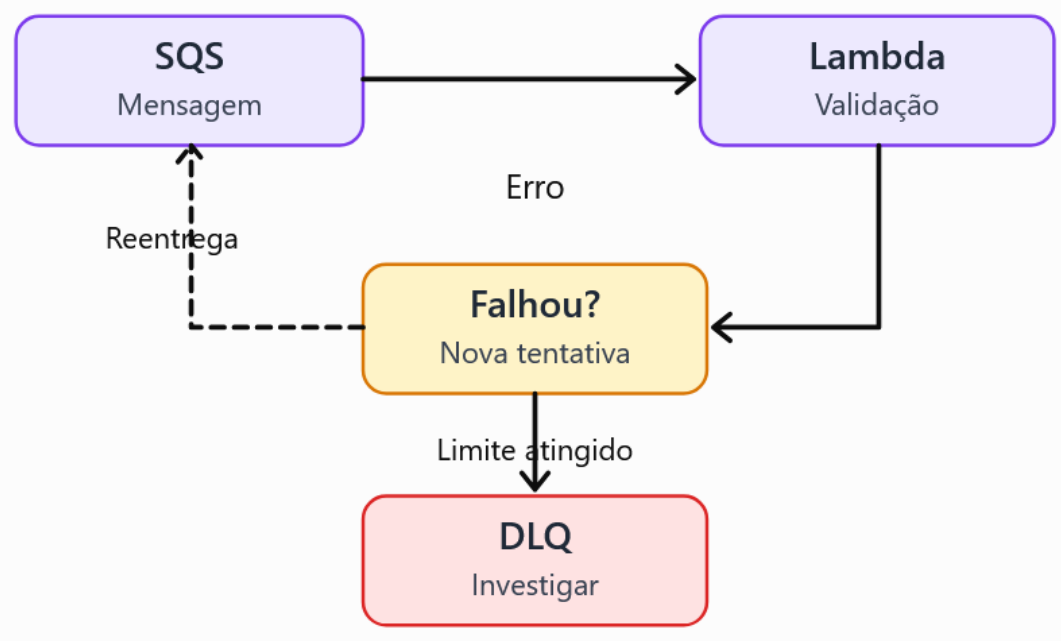
- A DLQ é uma fila separada, destinada a mensagens que não conseguiram ser processadas adequadamente após o número de tentativas permitido.
- Imagine uma mensagem recebida com um valor inválido:

```
{
  "device_id": "esp32-torno-01",
  "temperatura": "ERRO",
  "vibracao": 2.45,
  "rpm": 3480
}
```

- Se a Lambda exigir que temperatura seja numérica, essa mensagem poderá provocar uma falha de validação.

4. DLQ — Dead Letter Queue

- Em uma configuração com DLQ, o ciclo pode ser:



A mensagem problemática pode ser inspecionada, corrigida na origem e, quando apropriado, reprocessada.
Importante: a DLQ não corrige automaticamente os dados. Ela ajuda a preservar mensagens problemáticas para diagnóstico.

5. AWS Lambda — Processamento da telemetria

- A Lambda é o componente responsável por transformar as mensagens recebidas em dados que possam ser persistidos no InfluxDB.
- Suas responsabilidades são:
  - Consumir mensagens do SQS por event source mapping.
  - Validar os dados de telemetria.
  - Gravar as medições brutas no InfluxDB.

5. AWS Lambda — Processamento da telemetria

- **Exemplo de processamento**
  - A mensagem original enviada pelo ESP32 contém:


```
{
  "device_id": "esp32-torno-01",
  "timestamp": 1794144000,
  "temperatura": 72.8,
  "vibracao": 2.45,
  "rpm": 3480,
  "status": "OPERATIONAL",
  "pecas_produzidas": 15,
  "pecas_defeito": 1
}
```

5. AWS Lambda — Processamento da telemetria

- A Lambda pode executar estas operações conceituais:
  - Interpretar o JSON recebido da fila.
  - Verificar a presença dos campos obrigatórios.
  - Validar os tipos e os valores.
  - Preparar a medição temporal.
  - Enviar a medição ao InfluxDB.
  - Concluir o processamento da mensagem.
- A Lambda não precisa permanecer em execução continuamente. A integração com SQS pode acioná-la conforme existam mensagens disponíveis.

5. AWS Lambda — Processamento da telemetria

- Um detalhe arquitetural importante
  - Como o InfluxDB está em um container na EC2, a Lambda precisa ter conectividade de rede com esse serviço.
  - Precisaremos verificar no Terraform como foram configurados os endereços, as regras de Security Group e as permissões necessárias.
  - Também devemos considerar que SQS e Lambda trabalham com entrega pelo menos uma vez no cenário padrão. Portanto, uma mensagem pode ser processada mais de uma vez. O projeto deve prever idempotência ou deduplicação quando isso for necessário.

6. Como a arquitetura evoluiu

- Agora temos um caminho de ingestão bem definido, desde a geração da telemetria até a gravação no banco temporal.
- A principal ideia desta etapa é que o recebimento da telemetria não precisa ocorrer na mesma velocidade ou no mesmo momento do processamento. A fila SQS separa essas responsabilidades, enquanto a DLQ oferece um mecanismo para lidar com mensagens que falham repetidamente.

Etapa 3: Armazenamento e persistência de dados

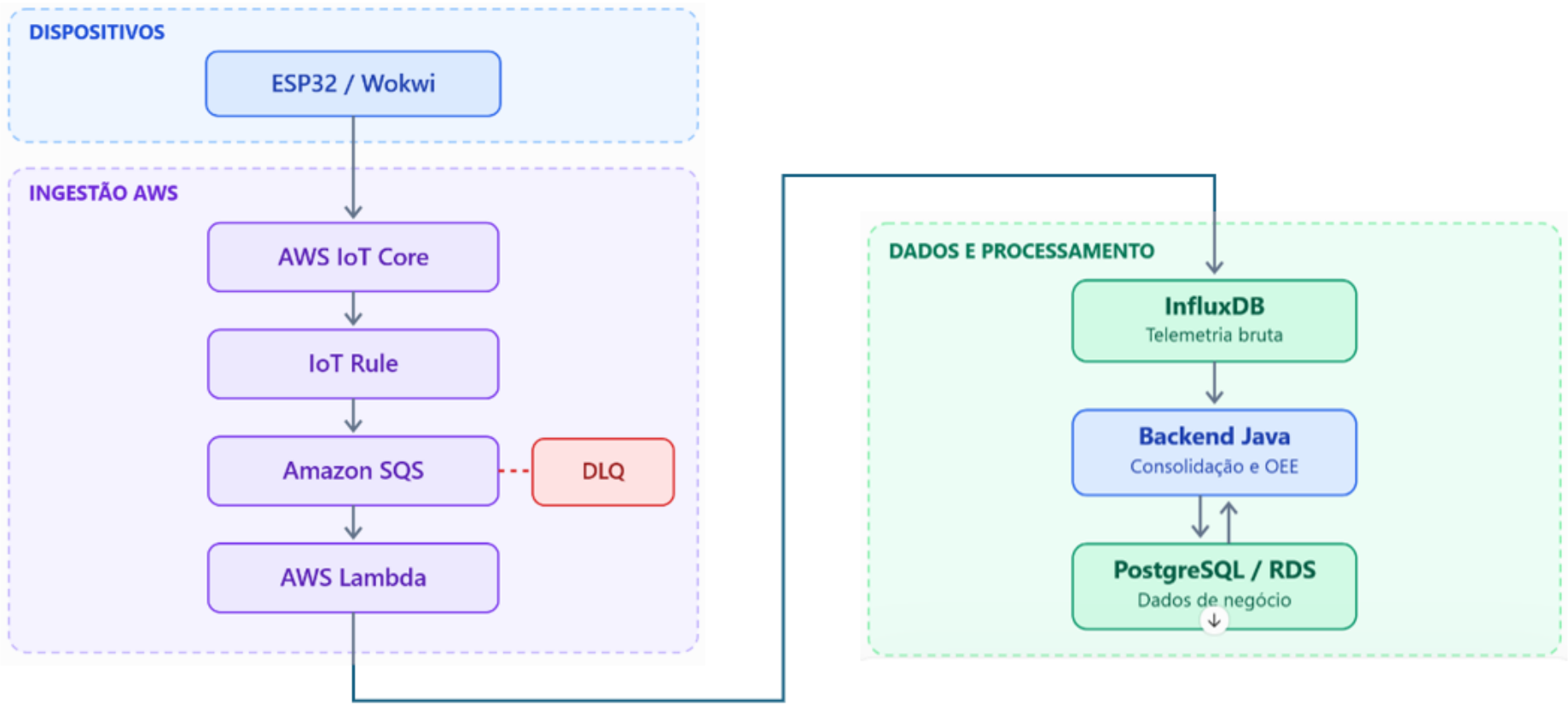
Nesta etapa, vamos compreender por que o Factory Hub utiliza dois bancos de dados diferentes, como as medições são armazenadas e de que maneira os dados brutos são transformados em informações gerenciais.

A principal decisão arquitetural é separar os dados de sensores, que chegam continuamente, dos dados de negócio, que precisam ser organizados e consultados.

1. Ampliando a arquitetura

- Até agora, construímos o caminho de ingestão:
- ESP32 → IoT Core → IoT Rule → SQS → Lambda
- Agora vamos acrescentar o InfluxDB, o PostgreSQL no Amazon RDS e o backend Java responsável pelas consolidações.

1. Ampliando a arquitetura



A mudança mais importante é que o InfluxDB recebe diretamente as medições processadas pela Lambda, enquanto o PostgreSQL recebe informações consolidadas pelo backend Java.

2. Por que utilizar dois bancos de dados?

- O Factory Hub utiliza uma arquitetura conhecida como persistência poliglota: diferentes tecnologias de armazenamento são utilizadas conforme a natureza dos dados.



2. Por que utilizar dois bancos de dados?

Característica	InfluxDB	PostgreSQL
Modelo	Séries temporais	Relacional
Dados principais	Leituras dos sensores	Cadastros e indicadores
Organização	Tempo, tags e fields	Tabelas e relacionamentos
Consultas típicas	Evolução e agregações temporais	Filtros, joins e relatórios
Implantação no projeto	Container Docker na EC2	Amazon RDS
Responsável pela escrita	Lambda	Backend Java

- Essa separação não significa que o PostgreSQL seja incapaz de armazenar séries temporais. É uma escolha de arquitetura para explorar características especializadas de cada tecnologia.

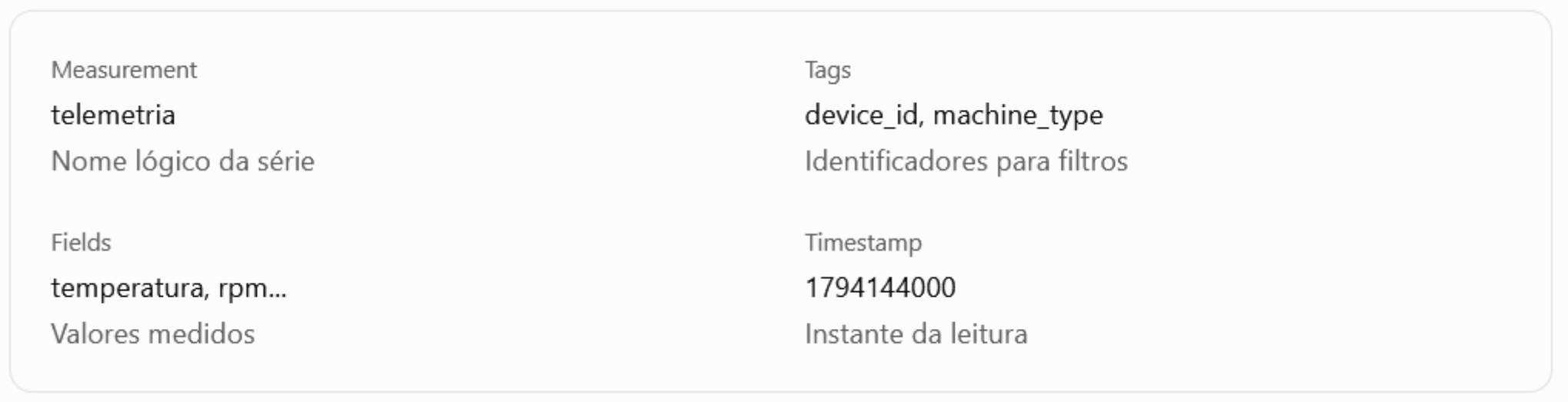
3. InfluxDB — Armazenando dados brutos

- Vamos acompanhar uma medição desde o ESP32.
- O dispositivo publica uma mensagem como esta:


```
{
  "device_id": "esp32-torno-01",
  "machine_type": "USINAGEM",
  "timestamp": 1794144000,
  "temperatura": 72.8,
  "vibracao": 2.45,
  "rpm": 3480,
  "status": "OPERATIONAL",
  "pecas_produzidas": 15,
  "pecas_defeito": 1
}
```

3. InfluxDB — Armazenando dados brutos

- **Como o InfluxDB organiza uma medição?**
- Uma medição típica possui quatro elementos:



Nesse exemplo, o timestamp foi convertido de segundos para nanossegundos, que é a precisão padrão do Line Protocol. Os contadores e o estado operacional também precisariam ser representados conforme o schema escolhido.

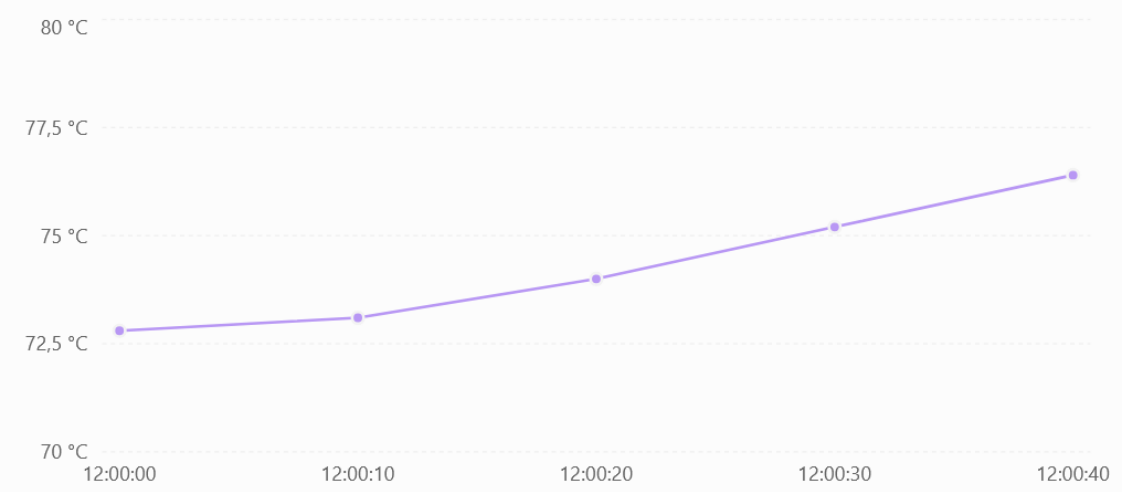
3. InfluxDB — Armazenando dados brutos

- Como ficam as medições ao longo do tempo?

Horário (UTC)	Temperatura	Vibração	RPM
12:00:00	72,8	2,45	3480
12:00:10	73,1	2,48	3500
12:00:20	74,0	2,62	3460
12:00:30	75,2	2,85	3410
12:00:40	76,4	3,10	3370

3. InfluxDB — Armazenando dados brutos

- Evolução da temperatura
- Dados ilustrativos de cinco medições de um equipamento



- O benefício é poder consultar o comportamento da máquina ao longo do tempo, em vez de conhecer apenas seu último estado.
- Esses dados também são úteis para a Random Forest da API Python, que utiliza o histórico do InfluxDB para treinamento.

4. Dados brutos versus dados consolidados

- Essa é uma das distinções mais importantes do projeto.
  - Dados brutos representam as leituras originais recebidas dos dispositivos.
  - Dados consolidados são resultados calculados a partir de várias leituras, agrupadas em um período.
- O projeto estabelece janelas de consolidação de cinco minutos, armazenadas no RDS.
- **Exemplo de consolidação**
  - Imagine que um equipamento envie uma medição a cada 10 segundos.
  - Em cinco minutos, teremos aproximadamente 30 medições.
  - O backend Java poderá calcular indicadores agregados a partir dessas leituras.

4. Dados brutos versus dados consolidados

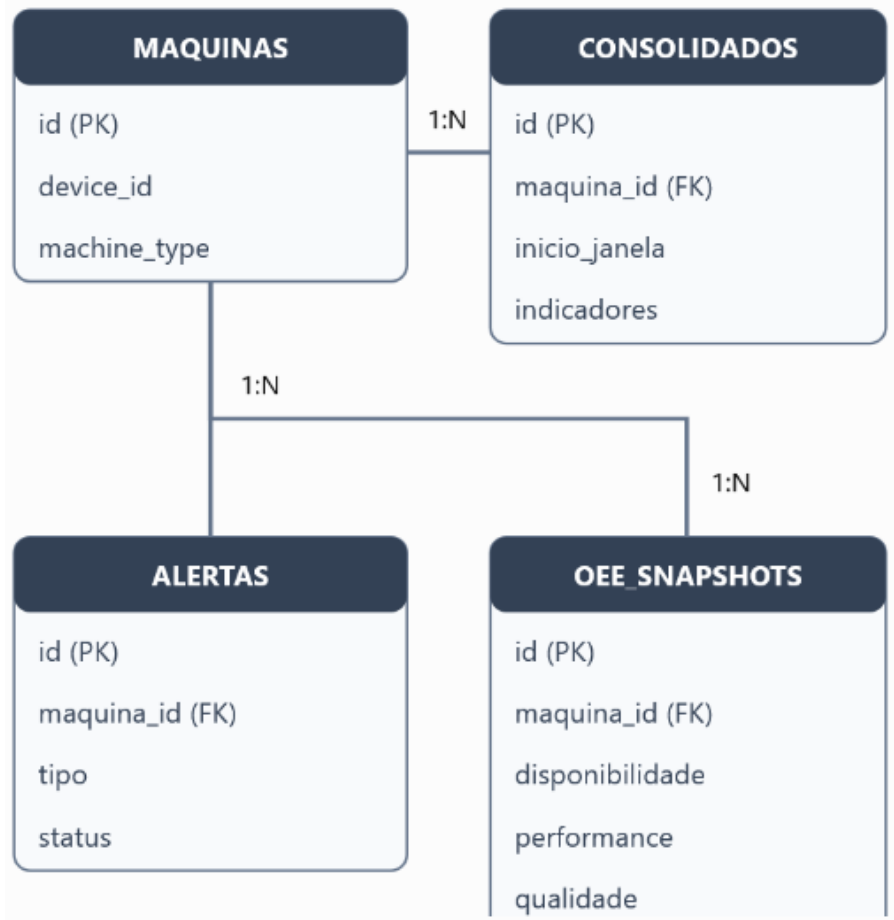
- No banco relacional, uma consolidação hipotética poderia ser armazenada assim:

device_id	Janela	Temp. média	Temp. máxima
esp32-torno-01	12:00–12:05	73,5	76,4
esp32-torno-01	12:05–12:10	74,2	77,1

- O ganho é que uma consulta gerencial não precisa necessariamente recuperar todas as leituras individuais para apresentar informações agregadas.

5. PostgreSQL no Amazon RDS

- O Amazon RDS é um serviço gerenciado de banco de dados da AWS.
- No Factory Hub, o PostgreSQL é utilizado para armazenar:
  - Cadastro de máquinas.
  - Alertas operacionais.
  - Snapshots didáticos de OEE.
  - Consolidações de telemetria em janelas de cinco minutos.
- Uma estrutura relacional conceitual poderia ser:

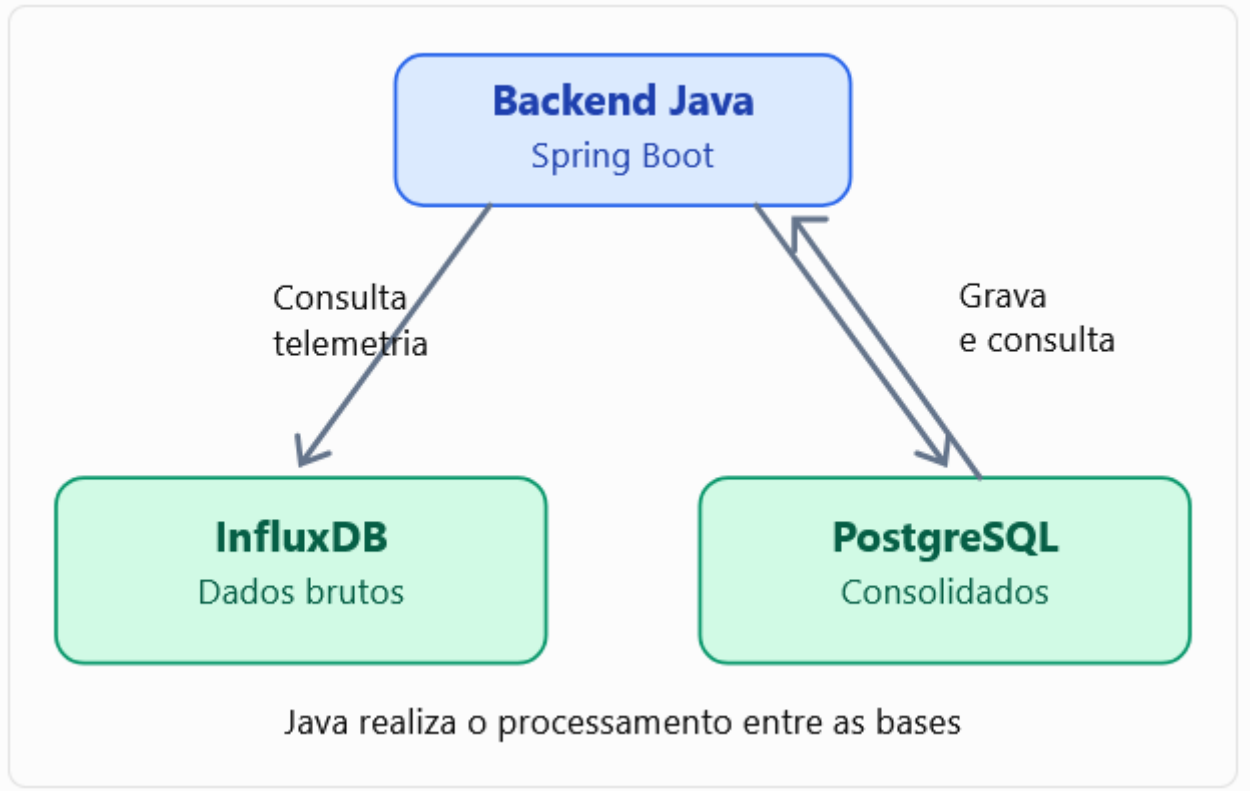


lodelo ilustrativo, não o DER efetivo do repositório. As chaves e relacionamentos precisam ser conferidos nas

6. Como o backend Java utiliza os dois bancos?

- O backend Java é o intermediário entre o armazenamento, as regras de negócio e a interface do usuário.
- Ele possui duas funções principais relacionadas aos bancos:
  - Leitura de telemetria: consulta as medições recentes do InfluxDB para disponibilizá-las ao frontend e calcular indicadores.
  - Persistência de negócio: utiliza o PostgreSQL para registrar e consultar máquinas, alertas e dados consolidados.

6. Como o backend Java utiliza os dois bancos?



O Java também calcula Disponibilidade, Performance e Qualidade para compor o OEE, desde que existam informações suficientes.

7. O que aprendemos nesta etapa

- principal conclusão é que o InfluxDB preserva o histórico operacional da fábrica, enquanto o PostgreSQL organiza os dados necessários à aplicação e à gestão.
- Essa separação prepara o Factory Hub para a próxima camada: processamento de negócio e inteligência artificial.

Etapa 4: Backend Java, API Python e Inteligência Artificial

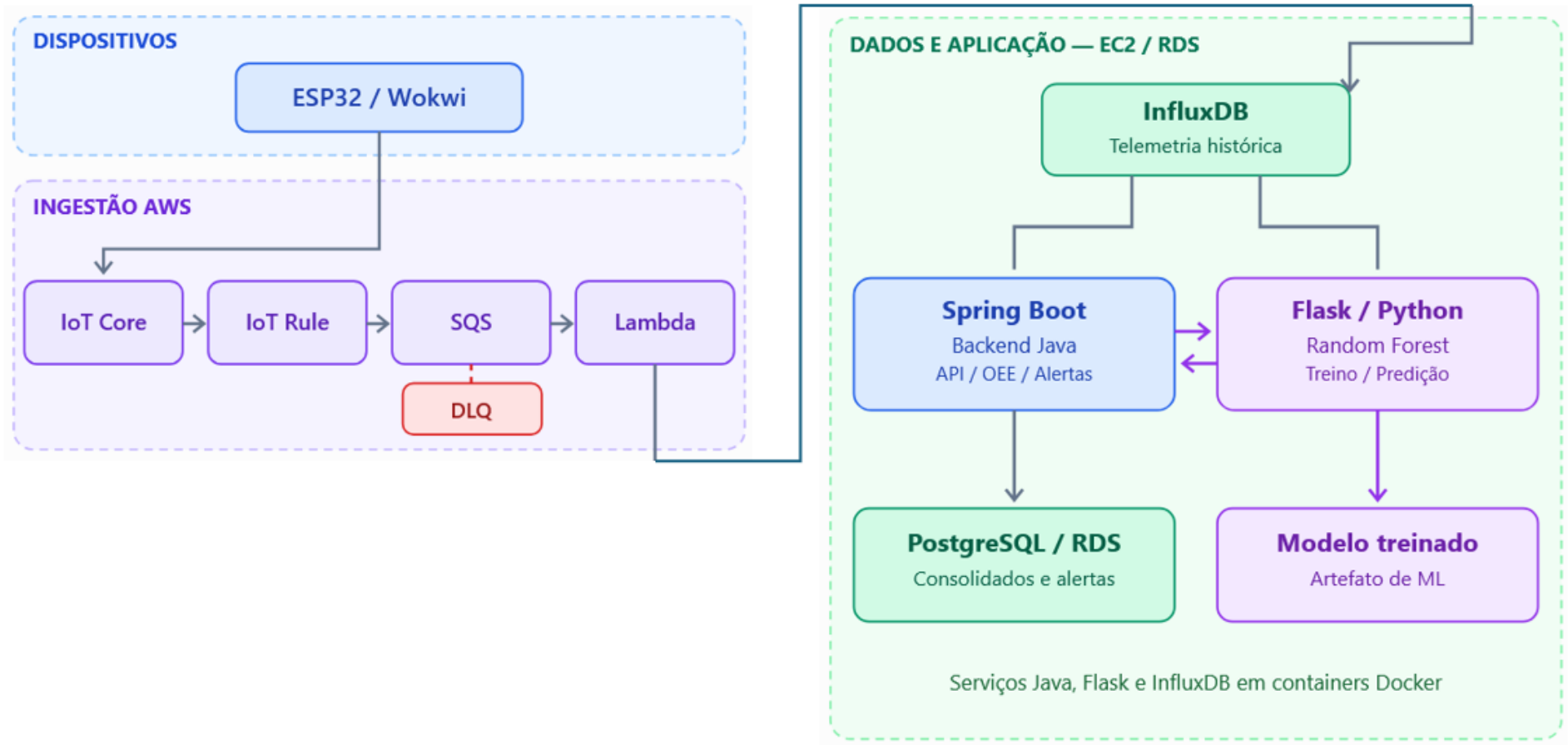
Até agora, construímos uma arquitetura capaz de receber telemetria industrial, processar mensagens e armazenar informações em dois bancos de dados.

Nesta etapa, vamos compreender como o Factory Hub transforma dados em indicadores, previsões e decisões operacionais.

Backend Java, API Python e Inteligência Artificial

- Entram em cena dois serviços com responsabilidades distintas:
  - Backend Java (Spring Boot): centraliza as regras de negócio, calcula indicadores, consulta os bancos e atende o frontend.
  - API Python (Flask + scikit-learn): utiliza dados históricos para treinar uma Random Forest e realizar previsões de anomalias.

1. Ampliando a arquitetura



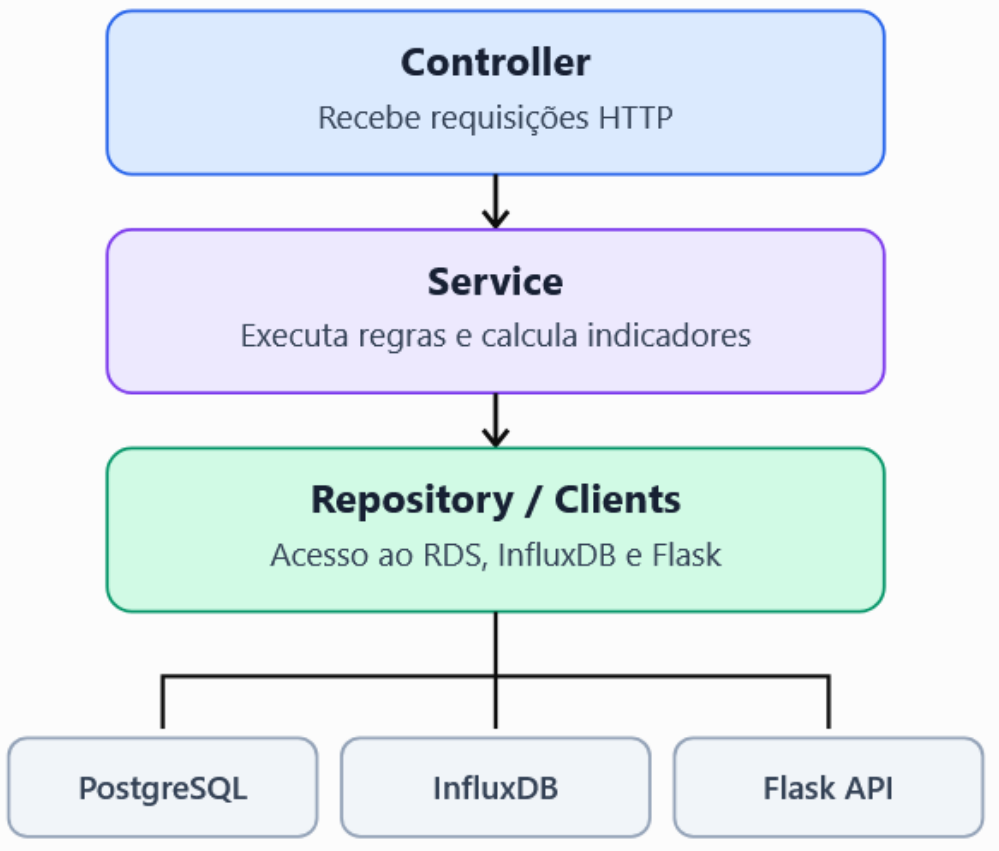
A arquitetura agora possui dois tipos de processamento: determinístico, realizado pelo Java por meio de regras e cálculos, e preditivo, realizado pelo Python com Machine Learning.

2. Backend Java — O cérebro das regras de negócio

- **Responsabilidades**
- O Java deve:
  - Consultar as últimas medições no InfluxDB.
  - Recuperar os cadastros e dados consolidados no PostgreSQL.
  - Consolidar telemetria em janelas de cinco minutos.
  - Calcular indicadores de Disponibilidade, Performance, Qualidade e OEE quando houver dados suficientes.
  - Intermediar solicitações de predição à API Python.
  - Disponibilizar endpoints REST para o frontend React.

2. Backend Java — O cérebro das regras de negócio

- Como o Java organiza suas responsabilidades?



3. API Python — O serviço de inteligência artificial

- Por que utilizar Python em um projeto que já possui Java?
- Porque o Python possui um ecossistema consolidado para ciência de dados. Separar a IA em um serviço permite desenvolver e atualizar o modelo sem misturar seu código às regras de negócio do backend Java.

3. API Python — O serviço de inteligência artificial

POST /predict

```
{
  "device_id": "esp32-torno-01",
  "temperatura": 72.8,
  "vibracao": 2.45,
  "rpm": 3480
}
```

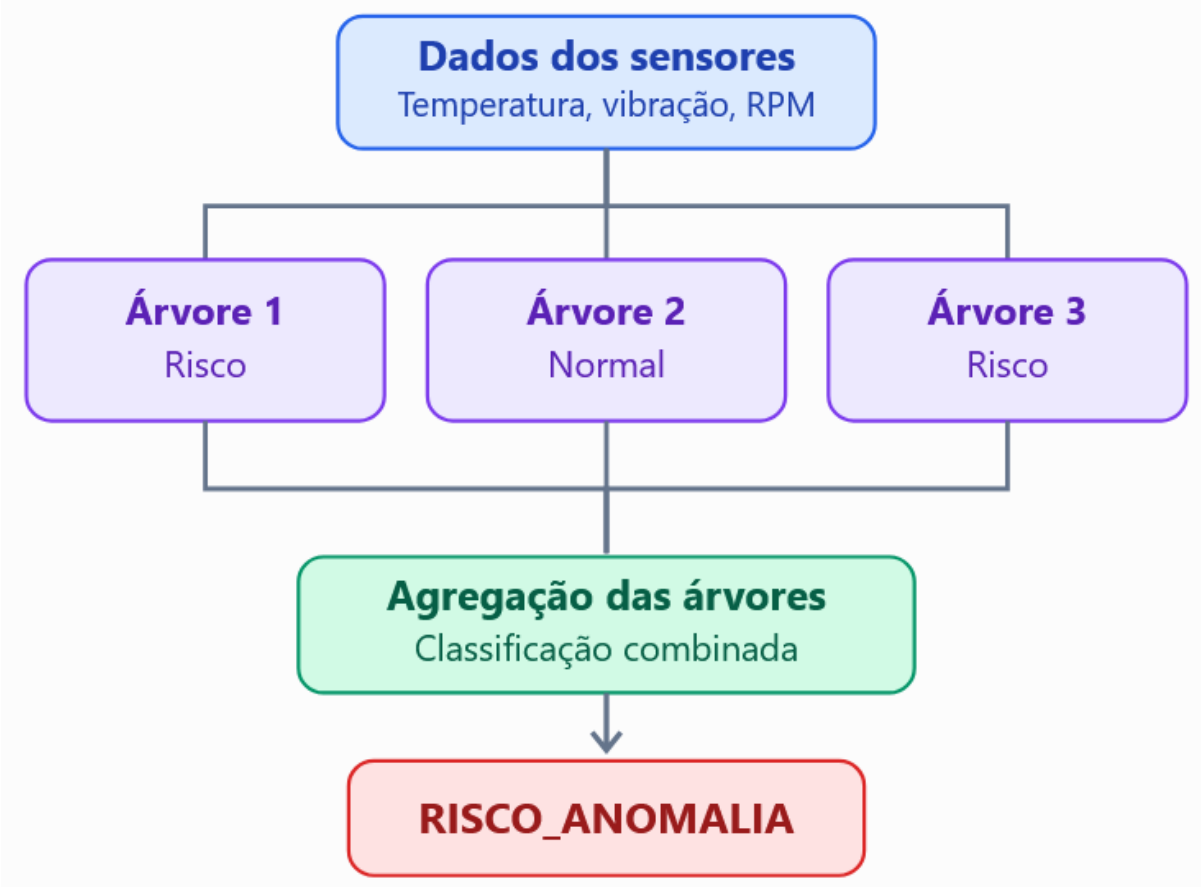
Exemplo de resposta documentada:

```
{
  "device_id": "esp32-torno-01",
  "status_previsto": "RISCO_ANOMALIA",
  "probabilidade_falha": 0.87,
  "recomendacao": "Reduzir carga e inspecionar rolamento principal",
  "modelo_versao": "mlp-v1.2"
}
```

Observe que o resultado contém mais do que uma classificação. Ele inclui uma probabilidade, uma recomendação e a versão do modelo.

4. Random Forest — Como funciona a predição?

- A Random Forest é um algoritmo de Machine Learning que combina várias árvores de decisão.
- Cada árvore aprende padrões a partir dos dados de treinamento e contribui para a decisão final.

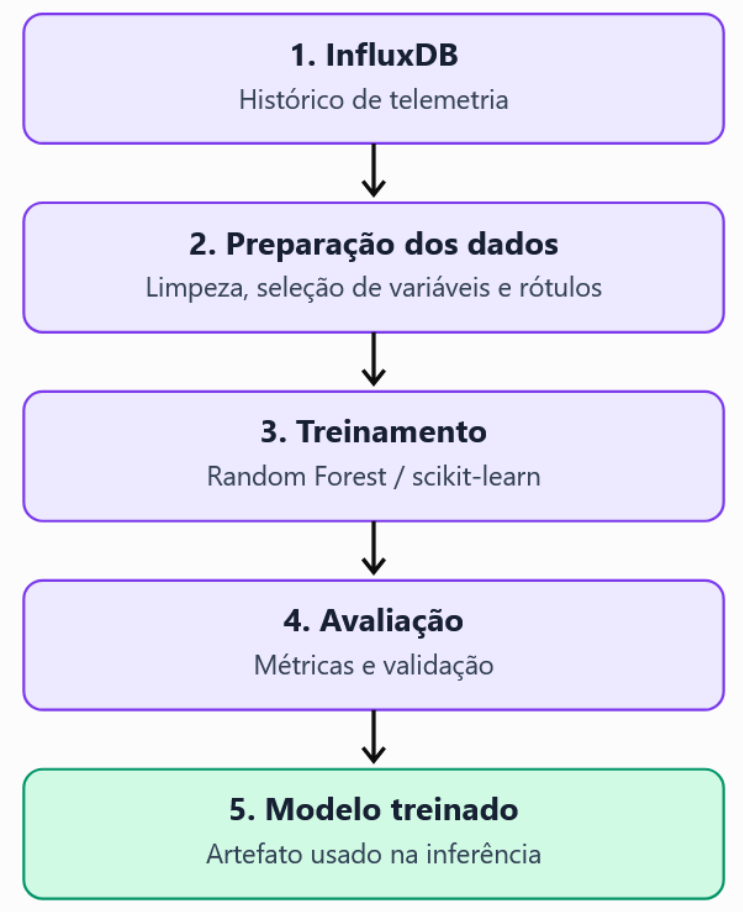


4. Random Forest — Como funciona a predição?

- Exemplo de decisão
- Uma árvore poderia aprender regras como:
  - Se a temperatura estiver elevada e a vibração também estiver elevada, classificar como risco.
  - Se a temperatura estiver normal e a vibração estiver baixa, classificar como operação normal.
  - Se o RPM estiver abaixo do padrão, considerar esse comportamento na classificação.
- Essas regras são apenas exemplos explicativos. Os limites efetivos são aprendidos pelo algoritmo a partir dos dados.

5. Treinamento — De onde vêm os dados?

O processo conceitual é:



5. Treinamento — De onde vêm os dados?

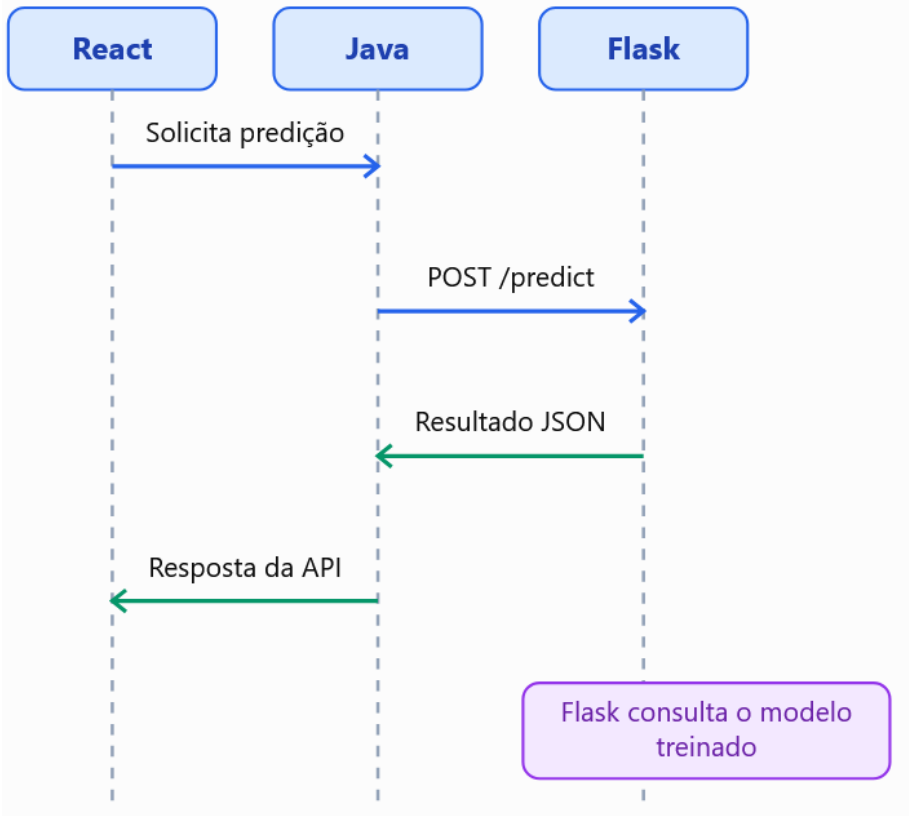
- Por que os rótulos são tão importantes?
  - A Random Forest, quando utilizada como classificador supervisionado, precisa de exemplos que indiquem o resultado esperado.

Temperatura	Vibração	RPM	Classe
68,2	1,20	3500	NORMAL
72,8	2,45	3480	NORMAL
89,1	5,80	2900	RISCO_ANOMALIA
94,5	7,10	2500	RISCO_ANOMALIA

- Se os rótulos forem gerados apenas por limites artificiais de temperatura e vibração, o modelo poderá aprender principalmente a reproduzir essas regras.
- Isso pode ser útil didaticamente, mas não comprova que ele consiga prever falhas industriais reais.

6. Como Java e Python se comunicam?

- O padrão arquitetural é uma comunicação HTTP entre serviços.

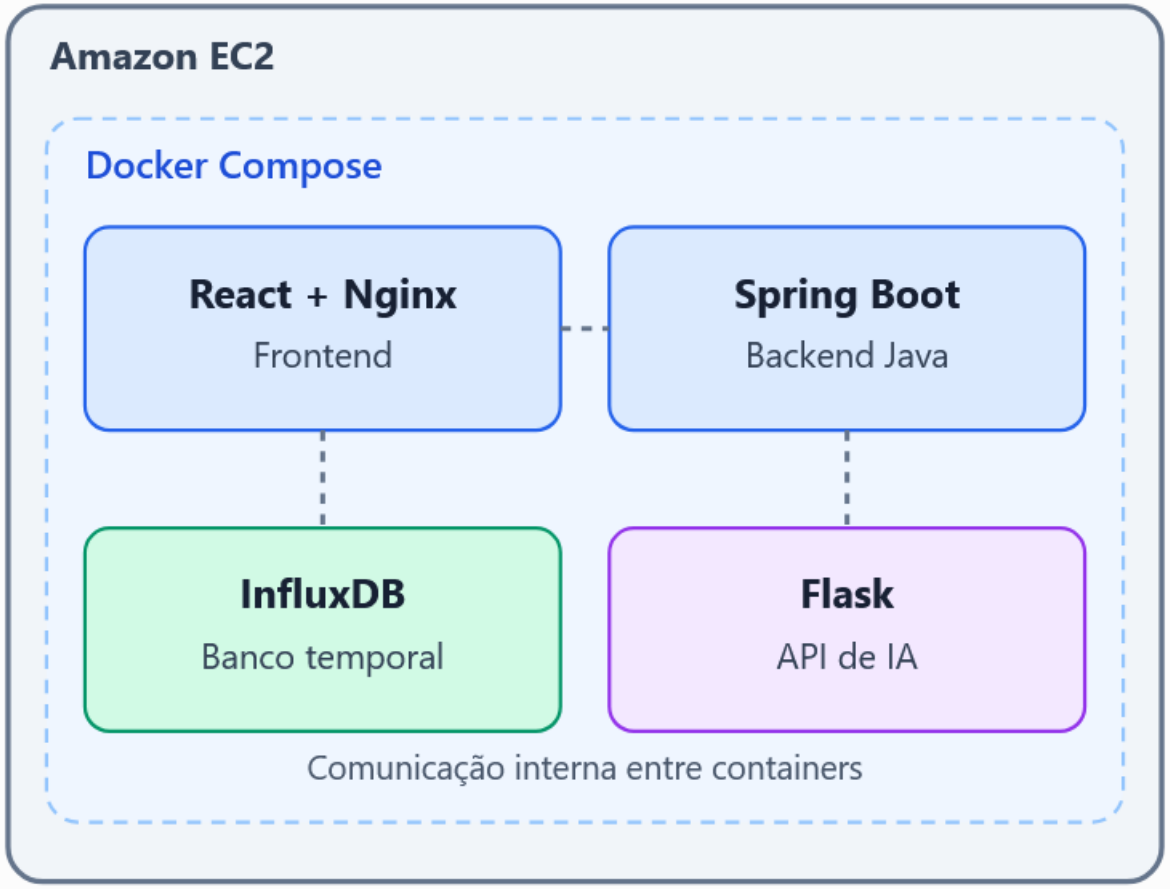


6. Como Java e Python se comunicam?

- Por que não permitir que o React chame diretamente o Flask?
  - A arquitetura proposta centraliza as regras e o acesso às funcionalidades no Java.
  - Isso oferece vantagens como controle de acesso, padronização das respostas e possibilidade de manter o Flask acessível apenas pela rede interna dos containers.
  - O Java funciona como uma camada de orquestração e integração, enquanto o Flask permanece especializado em Machine Learning.

7. Onde esses serviços são executados?

- Java, Flask, React e InfluxDB são executados em containers Docker na EC2.



8. Pontos de atenção na arquitetura

- A etapa de inteligência artificial acrescenta novos desafios.

Aspecto	Questão a validar
Treinamento	Os dados possuem rótulos confiáveis?
Avaliação	O modelo é testado com dados não utilizados no treinamento?
Versionamento	Como os modelos são armazenados e identificados?
Disponibilidade	O que acontece se o Flask estiver indisponível?
Segurança	O Flask é acessível apenas pelo backend Java?
Persistência	Os artefatos do modelo sobrevivem à recriação do container?
Alertas	Qual regra determina quando uma predição vira alerta?

- Um ponto especialmente importante é que detectar uma anomalia não equivale necessariamente a prever uma falha futura.
- Para uma solução de manutenção preditiva real, seria necessário validar se o modelo consegue antecipar falhas com antecedência útil, utilizando dados representativos e rótulos confiáveis.
- No contexto didático, a Random Forest pode demonstrar muito bem o fluxo completo de Machine Learning, desde a coleta até a inferência.

10. Evolução da arquitetura até aqui

A arquitetura já é capaz de representar o ciclo de dados da fábrica: captura → ingestão → armazenamento → processamento → predição.

Falta incorporar a experiência do usuário, por meio do dashboard React, e depois entender como toda essa infraestrutura é provisionada e atualizada.



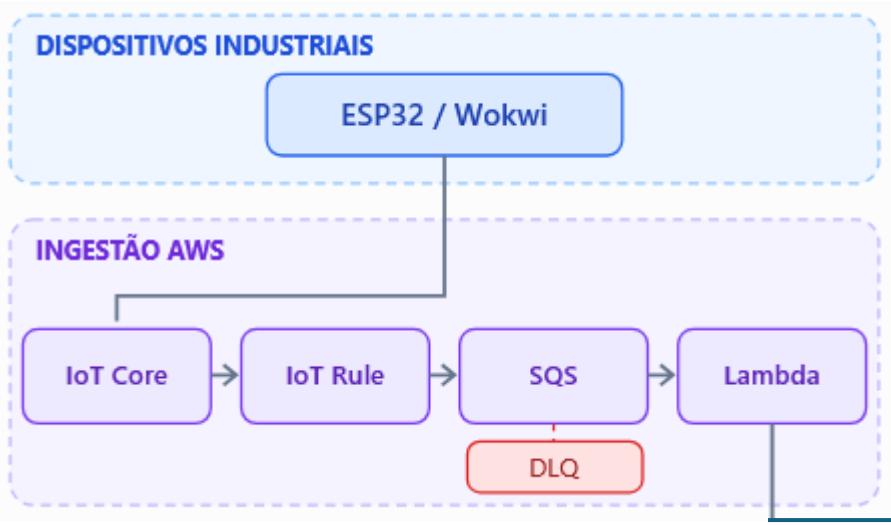
Etapa 5: Frontend React, Dashboards e Visualização

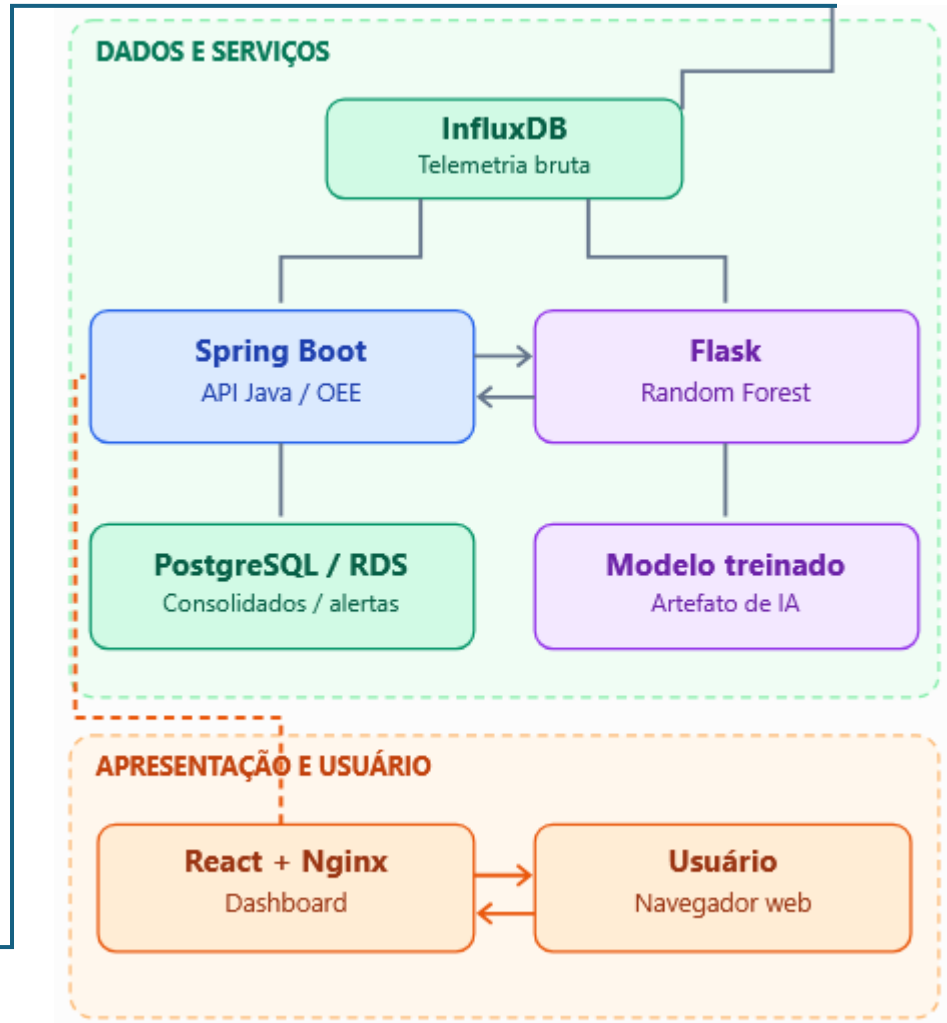
Até aqui, o Factory Hub já possui uma arquitetura capaz de receber dados industriais, armazenar telemetria, calcular indicadores e executar previsões utilizando inteligência artificial.

Agora vamos acrescentar a camada com a qual o usuário interage: o frontend React.

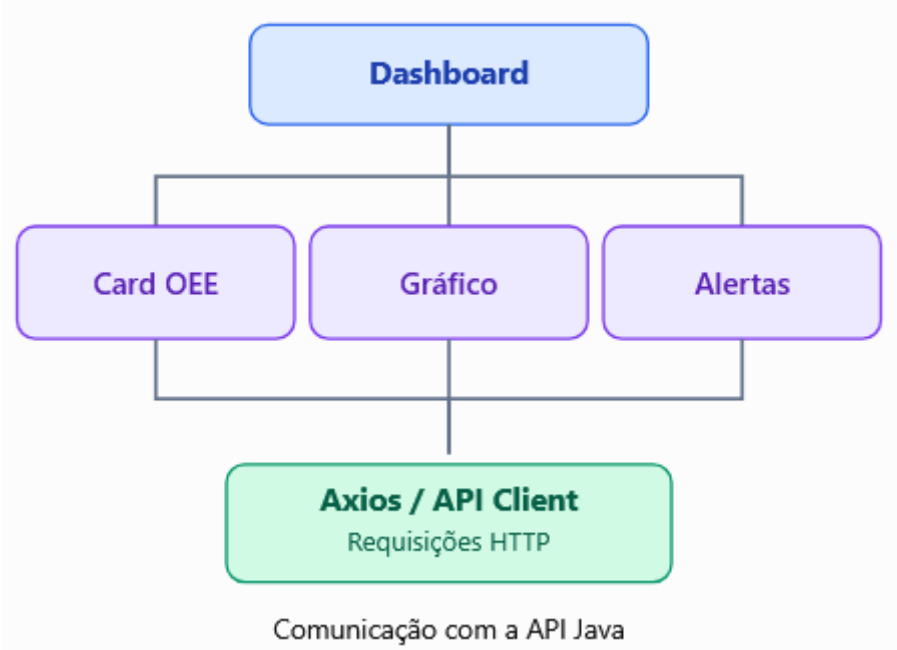
1. Ampliando a arquitetura — Incluindo o usuário

- Nesta etapa, completamos o fluxo funcional da aplicação, desde a máquina industrial até a visualização gerencial.





2. React — Construindo a interface do Factory Hub



Comunicação com a API Java

3. Axios — Comunicação entre React e Java

- O projeto estabelece que o React se comunica com o backend Java, não diretamente com os bancos de dados.
- **Exemplo de consulta**
- Para buscar as máquinas cadastradas:

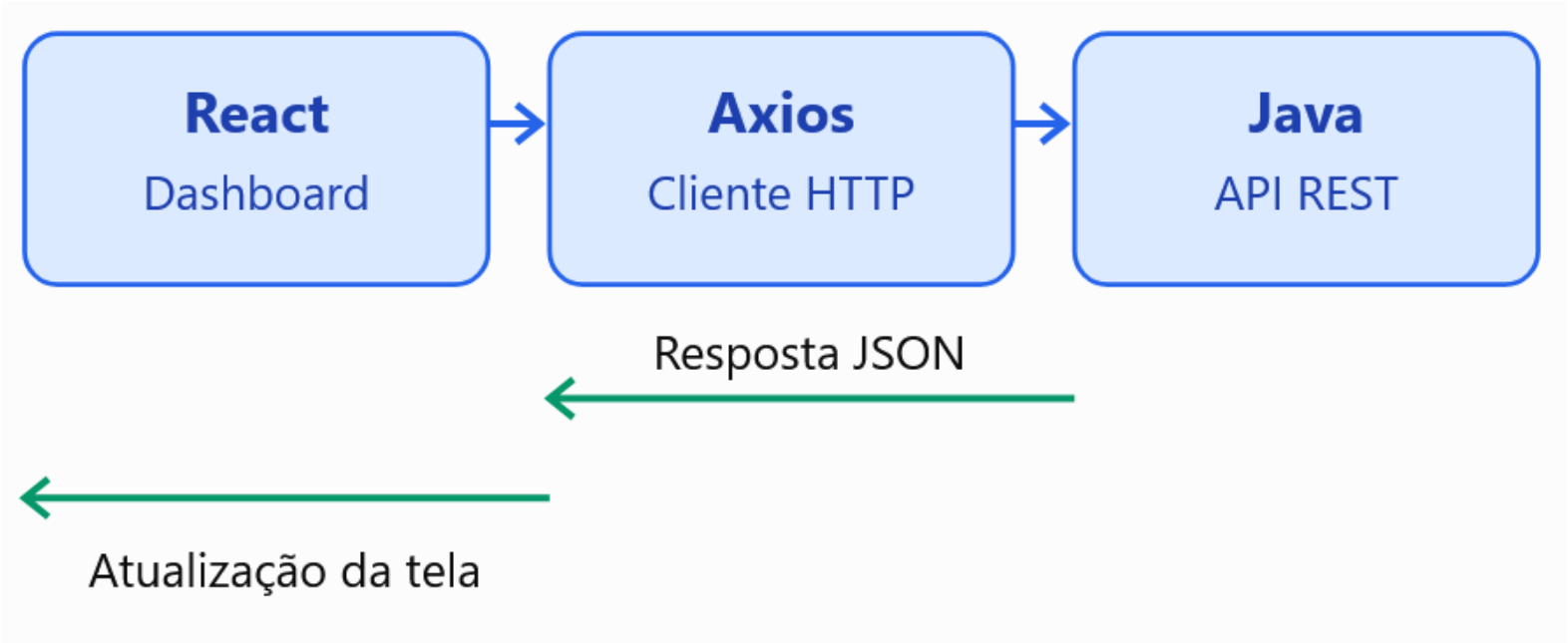
```
import axios from "axios";

const api = axios.create({
  baseURL: "/api/v1",
  timeout: 10000
});

export async function listarMaquinas() {
  const response = await api.get("/maquinas");
  return response.data;
}
```

3. Axios — Comunicação entre React e Java

- O fluxo da requisição é:



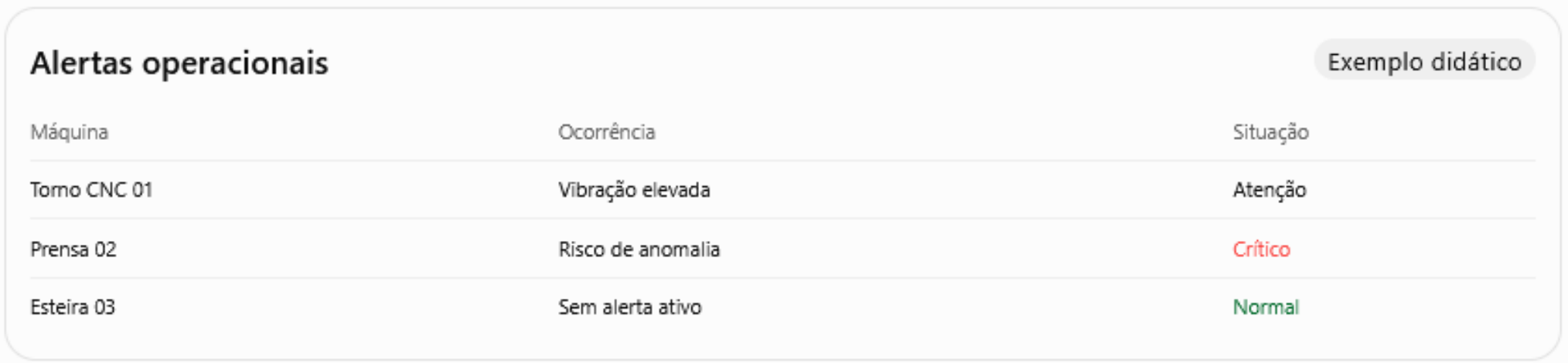
- O Axios não acessa diretamente o PostgreSQL, o InfluxDB ou o Flask. A API Java centraliza essas integrações.

4. OEE — Indicador de eficiência industrial

- O OEE (Overall Equipment Effectiveness) combina três fatores:
  - OEE = Disponibilidade $*$ Performance $*$ Qualidade
- **Disponibilidade**
  - Representa a proporção do tempo planejado em que a máquina esteve efetivamente operando.
  - $\mathrm{D} = \frac{\text{Tempo operacional}}{\text{Tempo planejado}}$
- **Performance**
  - Representa o desempenho da produção em relação ao ritmo ideal.
  - $P = \frac{\text{Produção real}}{\text{Produção teórica ideal}}$
- **Qualidade**
  - Representa a proporção de peças boas em relação ao total produzido.
  - $Q = \frac{\text{Peças boas}}{\text{Peças produzidas}}$

5. Alertas e previsões no dashboard

- A interface pode consultar esses registros por meio do endpoint:
- GET /api/v1/alertas



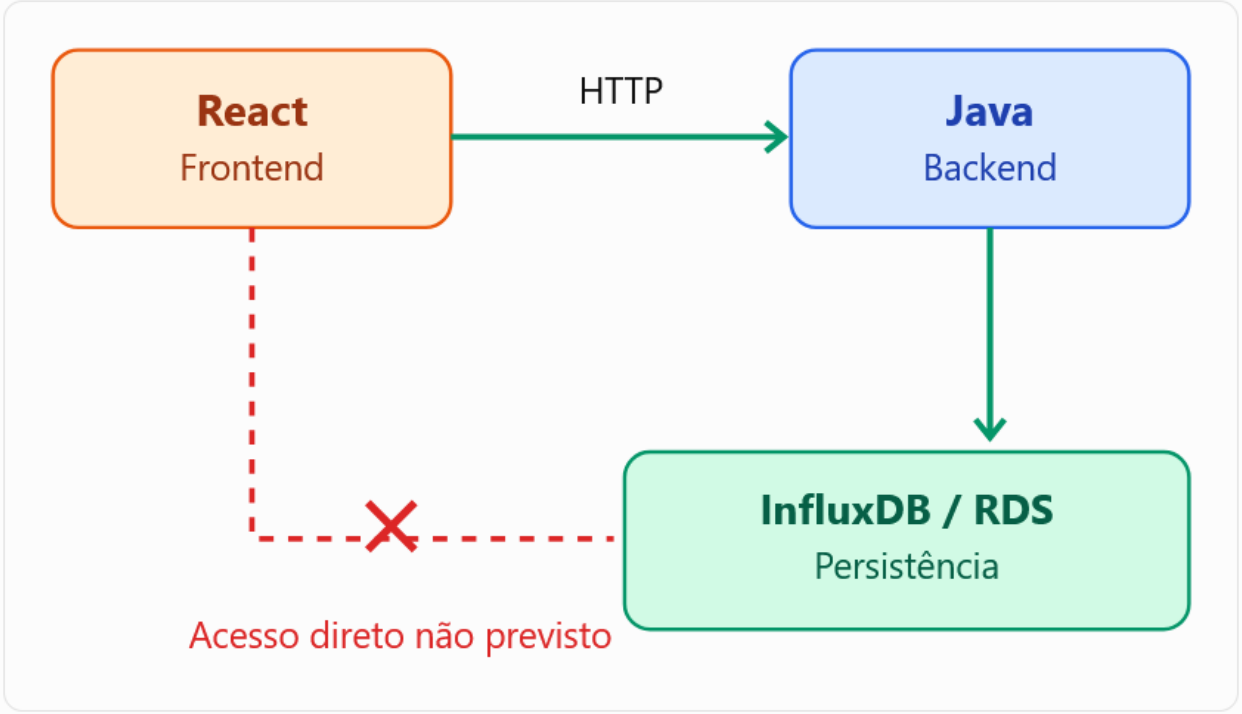
Alertas operacionais — Exemplo didático

Máquina	Ocorrência	Situação
Torno CNC 01	Vibração elevada	Atenção
Prensa 02	Risco de anomalia	Crítico
Esteira 03	Sem alerta ativo	Normal

- Uma previsão produzida pelo Flask pode originar um alerta de risco, conforme as regras de negócio implementadas no Java

6. O frontend deve conhecer os bancos de dados?

- Não. Uma decisão arquitetural importante do Factory Hub é manter a interface separada da persistência.



7. O que aprendemos nesta etapa

Com o frontend, concluímos a arquitetura funcional do Factory Hub. Agora o sistema possui todas as camadas necessárias para demonstrar a jornada da informação: da geração da telemetria no ESP32 até sua apresentação em um dashboard gerencial.



Etapa 6: Infraestrutura AWS, Docker, Terraform e DevOps

Uma característica importante do projeto é que provisionar infraestrutura e atualizar a aplicação são operações separadas, com workflows próprios.

1. Arquitetura final — Três perspectivas

- Para compreender a solução completa, vamos separar a arquitetura em três diagramas complementares.



- Essa separação é importante porque a arquitetura de software não é exatamente a mesma coisa que a arquitetura de infraestrutura.

2. Amazon EC2 — Onde a aplicação é executada

Container	Responsabilidade
React + Nginx	Servir o frontend
Java + Spring Boot	API e regras de negócio
Python + Flask	Treinamento e predição
InfluxDB	Armazenamento de telemetria

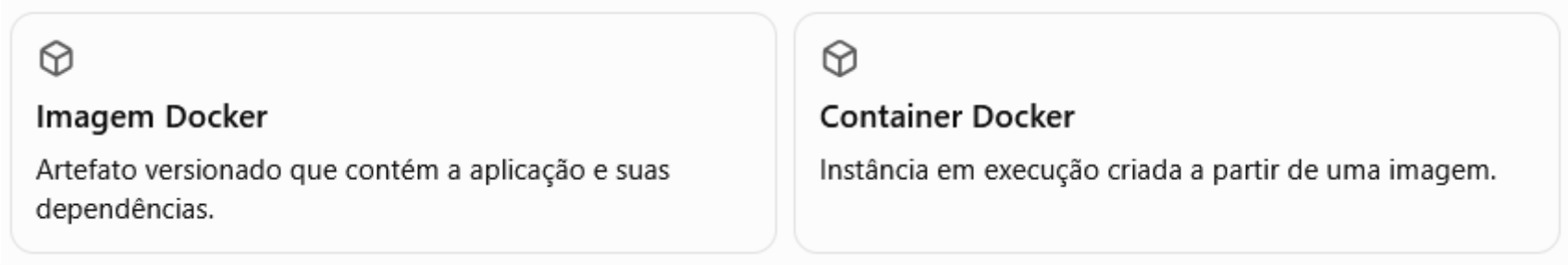
O PostgreSQL não é executado dentro da EC2. Ele utiliza o Amazon RDS.

2. Amazon EC2 — Onde a aplicação é executada

- Por que utilizar uma EC2 com vários containers?
  - No contexto didático do AWS Academy Learner Lab, essa estratégia reduz a quantidade de recursos de computação necessários e simplifica a implantação.
  - Também permite que os alunos estudem containerização, redes Docker, integração entre serviços e deploy remoto sem precisar implementar uma plataforma de orquestração mais complexa, como Kubernetes.
  - A principal limitação é que os serviços compartilham os recursos e a disponibilidade de uma única máquina virtual.
  - Se a EC2 ficar indisponível, o frontend, o Java, o Flask e o InfluxDB poderão ser afetados simultaneamente.

3. Docker — Isolamento dos serviços

- Cada componente do Factory Hub possui necessidades diferentes.
- O Java utiliza uma JVM e dependências Maven. O Flask utiliza Python e bibliotecas de Machine Learning. O React gera arquivos estáticos servidos pelo Nginx. O InfluxDB possui seu próprio processo de armazenamento.
- O Docker permite executar esses ambientes separadamente.
- **Imagem versus container**

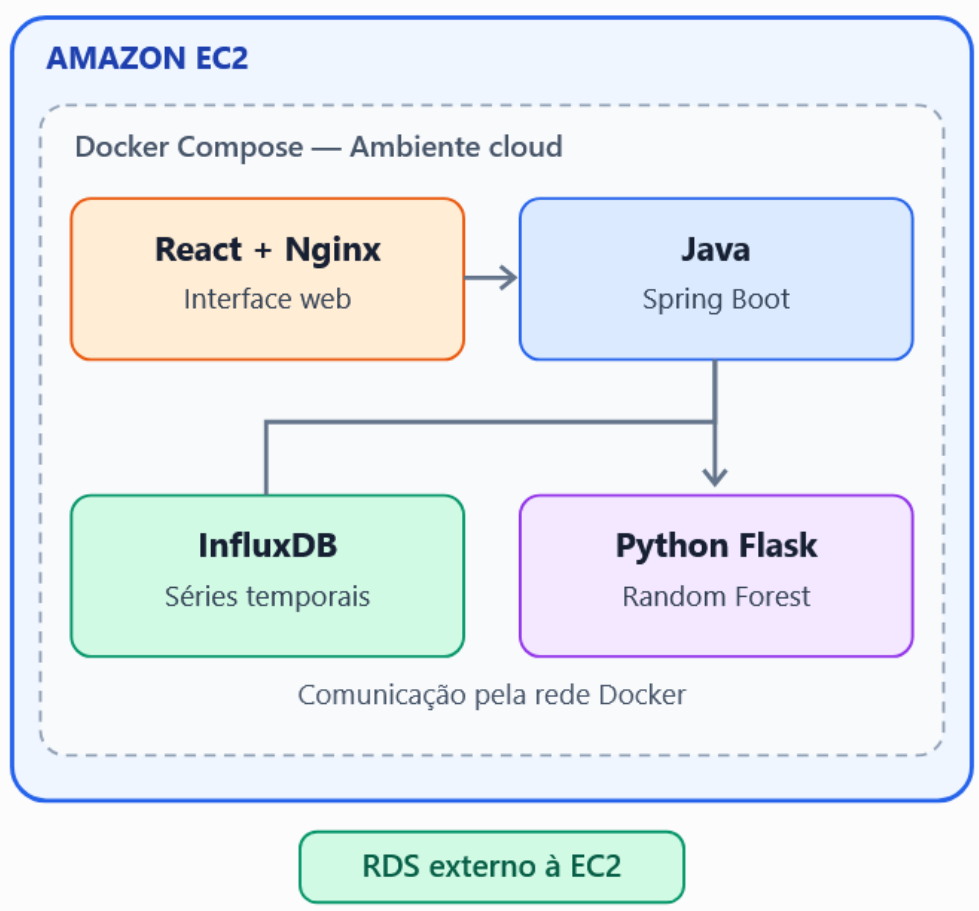


- No Factory Hub, as imagens são construídas e publicadas no Docker Hub. A EC2 baixa as imagens correspondentes às versões desejadas.

4. Docker Compose — Organizando os containers

- *docker-compose.yml*: orquestração local.
- *docker-compose.cloud.yml*: execução da aplicação na EC2.
- O Docker Compose define os serviços que serão executados em conjunto, suas configurações, redes, volumes e dependências.

4. Docker Compose — Organizando os containers



5. Amazon RDS — Banco de dados gerenciado

- A separação entre EC2 e RDS é importante.
- A EC2 executa as aplicações, enquanto o RDS gerencia a instância do banco relacional. Assim, uma atualização dos containers Java ou Flask não exige recriar o PostgreSQL.
- O README também estabelece que o RDS é privado.
- Isso significa que sua conectividade deve ser controlada pela rede AWS e pelas regras de segurança, em vez de disponibilizar o banco diretamente na internet.
- O RDS não elimina a necessidade de configurar backups, armazenamento, credenciais e disponibilidade adequadamente.

6. Redes AWS — VPC, sub-redes e roteamento

Para compreender a comunicação entre os serviços, precisamos entender a infraestrutura de rede.

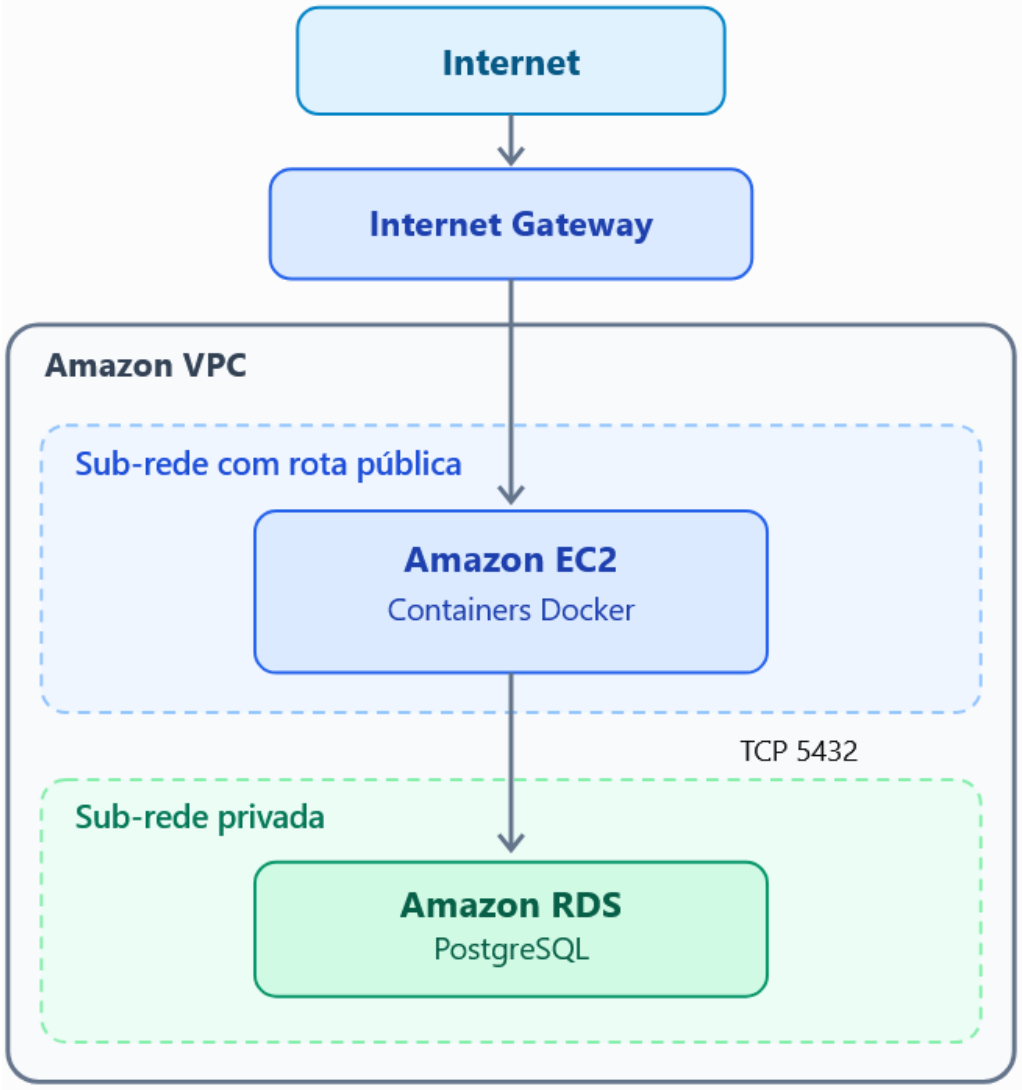
VPC — Virtual Private Cloud
Uma VPC é uma rede virtual isolada logicamente dentro da AWS.
Nela podemos definir endereçamento IP, sub-redes, tabelas de rotas e regras de acesso.

Sub-redes públicas e privadas
Uma arquitetura típica utiliza:

Sub-rede	Finalidade
Pública	Hospedar recursos que precisam de uma rota para a internet, como uma EC2 com frontend público
Privada	Hospedar recursos que não devem receber conexões diretamente da internet, como o RDS

6. Redes AWS — VPC, sub-redes e roteamento

- Uma sub-rede pública é definida por sua rota para um Internet Gateway; isso, por si só, não torna todos os recursos nela acessíveis publicamente.



6. Redes AWS — VPC, sub-redes e roteamento

- **E os serviços IoT Core, SQS e Lambda?**
  - IoT Core e SQS são serviços gerenciados da AWS e não são simplesmente containers ou servidores instalados nas sub-redes da aplicação.
  - A Lambda também é gerenciada. Dependendo da configuração, pode receber acesso à VPC para alcançar recursos privados, como o InfluxDB hospedado na EC2.
  - Isso introduz uma decisão importante: como a Lambda acessa o InfluxDB sem expor desnecessariamente sua porta à internet?

7. Security Groups — Controlando o tráfego

- Os Security Groups funcionam como firewalls virtuais associados a recursos de rede da AWS.
- Eles definem quais conexões de entrada e saída são permitidas.
- São stateful: o tráfego de resposta de uma conexão permitida é acompanhado automaticamente.

7. Security Groups — Controlando o tráfego

- **Matriz conceitual de comunicação**

Origem	Destino	Protocolo/porta	Finalidade
Navegador	Frontend EC2	HTTPS / 443	Acesso ao dashboard
Frontend/API proxy	Java	HTTP interno	Requisições REST
Java	PostgreSQL RDS	TCP / 5432	Consultas SQL
Java e Flask	InfluxDB	Porta interna configurada	Consultas temporais
Java	Flask	HTTP interno	Solicitação de predição
Lambda	InfluxDB	Porta configurada	Gravação de telemetria
EC2	Docker Hub	HTTPS / 443	Download das imagens

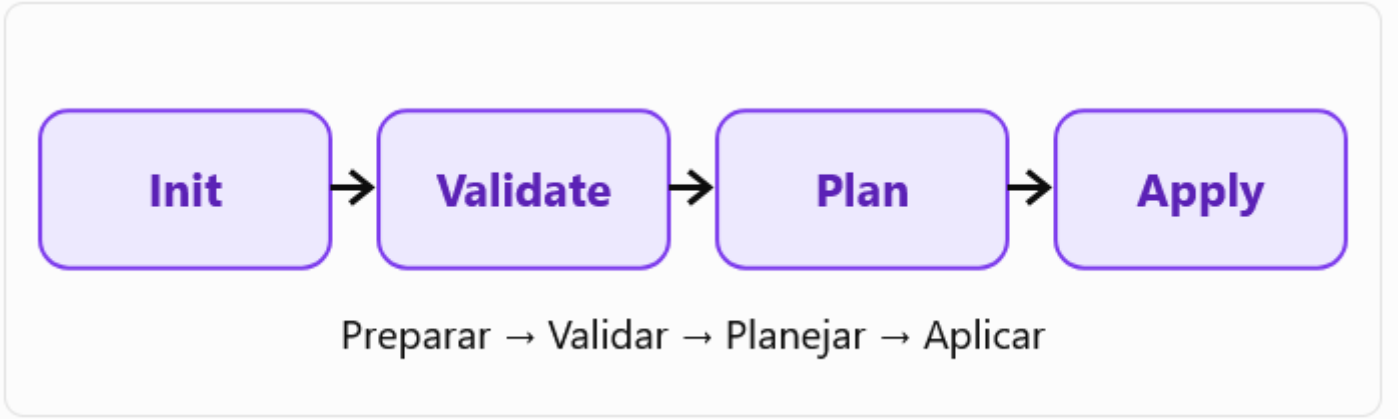
8. Terraform — Infraestrutura como código

- Em vez de criar manualmente cada recurso pelo console AWS, o Terraform descreve o estado desejado da infraestrutura.
- No Factory Hub apresenta esta organização:

Arquivo	Responsabilidade
backend.tf	Configuração do state remoto S3
main.tf	Região, AMI e dados de configuração
network.tf	VPC, sub-redes, rotas e Internet Gateway
compute.tf	EC2, Docker e SSM
storage.tf	RDS PostgreSQL privado
ingestion.tf	IoT Rule, SQS, Lambda e integração
security_groups.tf	Regras de acesso
terraform.tfvars.example	Exemplo de variáveis e perfis IAM

8. Terraform — Infraestrutura como código

- O ciclo de execução



- terraform init inicializa o diretório e configura os providers e o backend.
- terraform validate verifica a consistência sintática e estrutural.
- terraform plan mostra as mudanças previstas.
- terraform apply executa as mudanças na infraestrutura.

8. Terraform — Infraestrutura como código

- **O state remoto no S3**
  - O Terraform mantém um arquivo de estado que relaciona os recursos declarados aos recursos reais.
  - No Factory Hub, esse estado é armazenado em um bucket Amazon S3, indicado pela variável TF_STATE_BUCKET.
- E necessário configurar previamente:
  - Bloqueio de acesso público.
  - Versionamento.
  - Criptografia.
  - Variáveis e credenciais IAM necessárias.
- O state deve ser protegido porque pode conter informações sensíveis da infraestrutura. Também é importante verificar como o workflow impede alterações simultâneas no mesmo state.

8. Terraform — Infraestrutura como código

- Particularidade do AWS Academy Learner Lab
  - O projeto informa que as credenciais AWS são temporárias e ficam configuradas nos GitHub Secrets.
  - Isso significa que o provisionamento depende de credenciais válidas no momento da execução.
  - A renovação dessas credenciais e as permissões disponíveis são aspectos operacionais importantes do laboratório.

9. GitHub Actions — Automatizando o projeto

O projeto usa GitHub Actions para validar alterações, publicar imagens Docker, fazer deploy na EC2 e provisionar a infraestrutura AWS. Cada etapa tem um workflow próprio, e publicação, deploy e Terraform são iniciados manualmente.

Workflow	Quando executa	O que faz
ci.yml	Em pull requests, pushes para main ou develop e execuções manuais	Chama os quality gates para verificar o código.
quality-gates.yml	Chamado pelo CI e pelo workflow de publicação	Executa lint de Python e React, testes Flask, Java, Lambda e frontend, testes E2E e builds das imagens Docker sem publicá-las.
publish-dockerhub.yml	Execução manual; publica somente quando a referência selecionada é main	Repete os quality gates e, se todos passarem, publica as imagens Flask, Java e React no Docker Hub com as tags latest e sha-<commit>.
deploy-ssm.yml	Execução manual, informando o SHA completo de 40 caracteres de um commit já publicado	Envia o comando de deploy à EC2 pelo AWS Systems Manager (SSM), para atualizar as imagens e a configuração da aplicação.
terraform.yml	Execução manual	Inicializa o Terraform com state remoto no S3 e executa validação, plan e apply para provisionar a infraestrutura AWS.

9. GitHub Actions — Automatizando o projeto

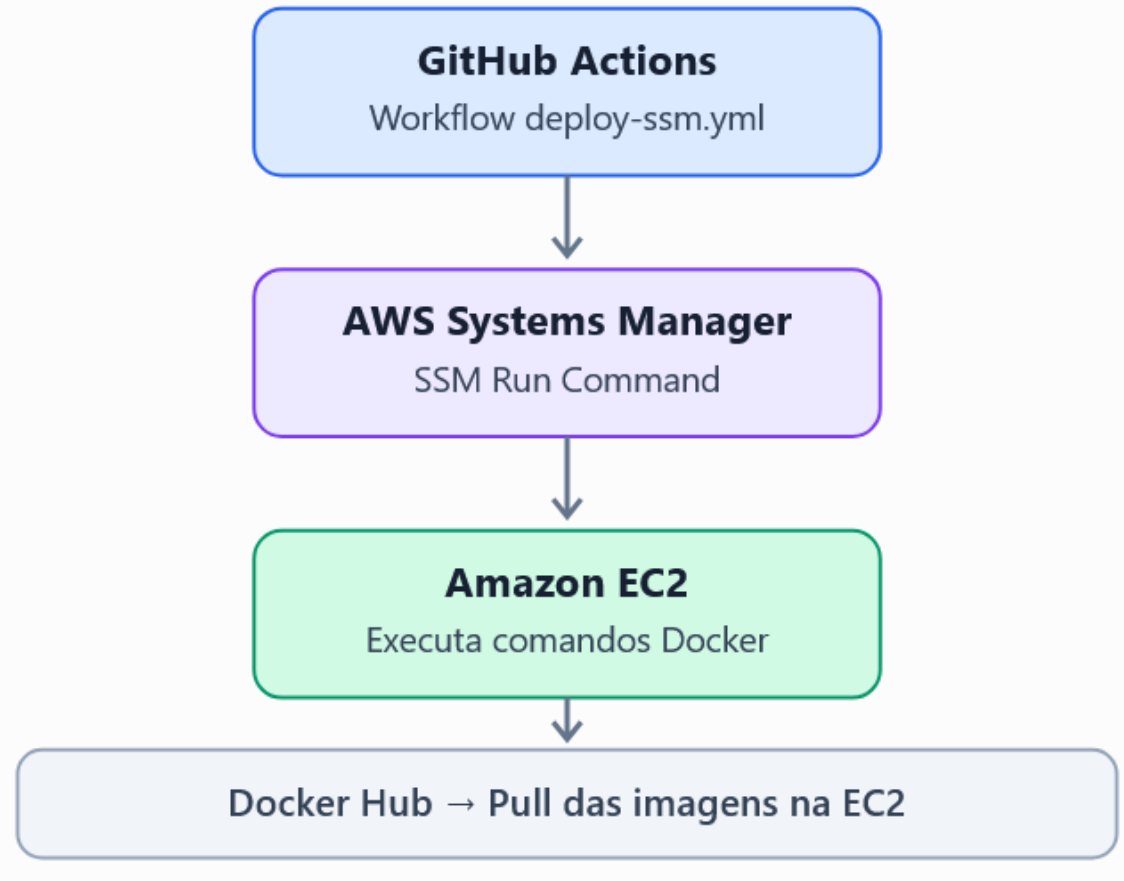
- **CI — Integração contínua**
  - O workflow ci.yml roda automaticamente em pull requests e em pushes para main e develop. Ele chama quality-gates.yml, que executa as verificações definidas para o projeto: lint, testes, testes E2E e builds Docker.
  - Os builds de CI verificam se as imagens podem ser construídas, mas não as publicam. A publicação no Docker Hub ocorre em um fluxo separado.
- **Publicação e deploy**
  - Para publicar uma versão, inicie manualmente publish-dockerhub.yml na branch main. Depois que os quality gates passarem, o workflow publica as três imagens com tags latest e sha-<commit>.
  - A publicação não atualiza a EC2. Para fazer o deploy, inicie deploy-ssm.yml e informe o SHA completo do commit publicado. O workflow usa o SSM para enviar comandos à instância; não é necessário abrir a porta SSH (22).
- **Provisionamento da infraestrutura**
  - O workflow terraform.yml também é iniciado manualmente. Ele provisiona ou atualiza recursos AWS usando as credenciais e configurações armazenadas nos GitHub Secrets e Repository variables. O apply faz parte da execução do workflow: iniciá-lo pode criar ou alterar recursos e gerar custos.
- Assim, o fluxo comum de uma atualização é: **CI automático → publicação manual → deploy manual via SSM**. O Terraform é executado separadamente quando há necessidade de provisionar ou alterar a infraestrutura.

10. Docker Hub — Registro das imagens

- O Factory Hub, o GitHub Actions constrói as imagens e as publica no Docker Hub.
- A EC2 posteriormente baixa essas imagens para executar os containers.
- **Exemplo de versionamento**
- Imagens hipotéticas do Factory Hub
  - factoryhub-frontend:1.2.0
  - factoryhub-backend:1.2.0
  - factoryhub-ml:1.2.0
- O uso de tags versionadas permite identificar qual versão está implantada e facilita um eventual rollback.
- Uma prática recomendável é também registrar o digest da imagem, pois uma tag pode ser sobrescrita.

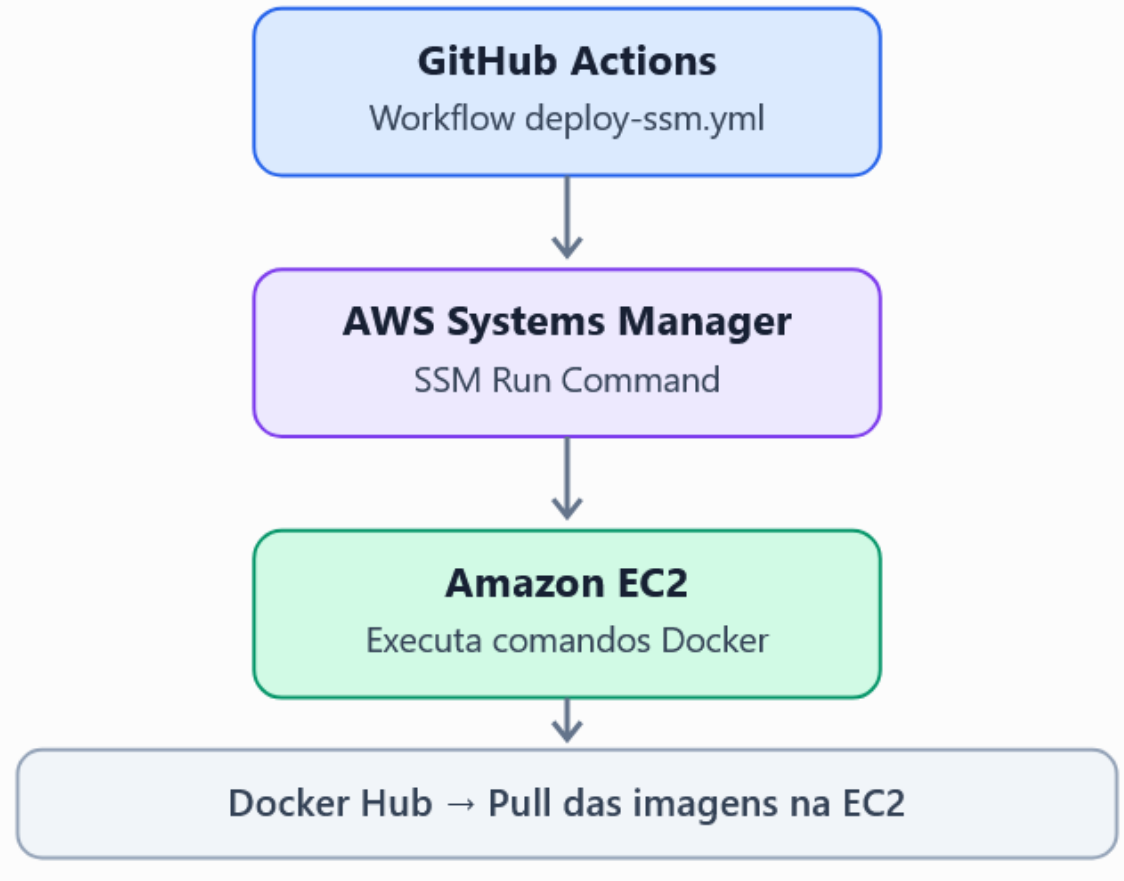
11. AWS Systems Manager — Deploy sem SSH

- Isso permite enviar comandos para a EC2 por meio do AWS Systems Manager, sem precisar abrir uma sessão SSH tradicional.

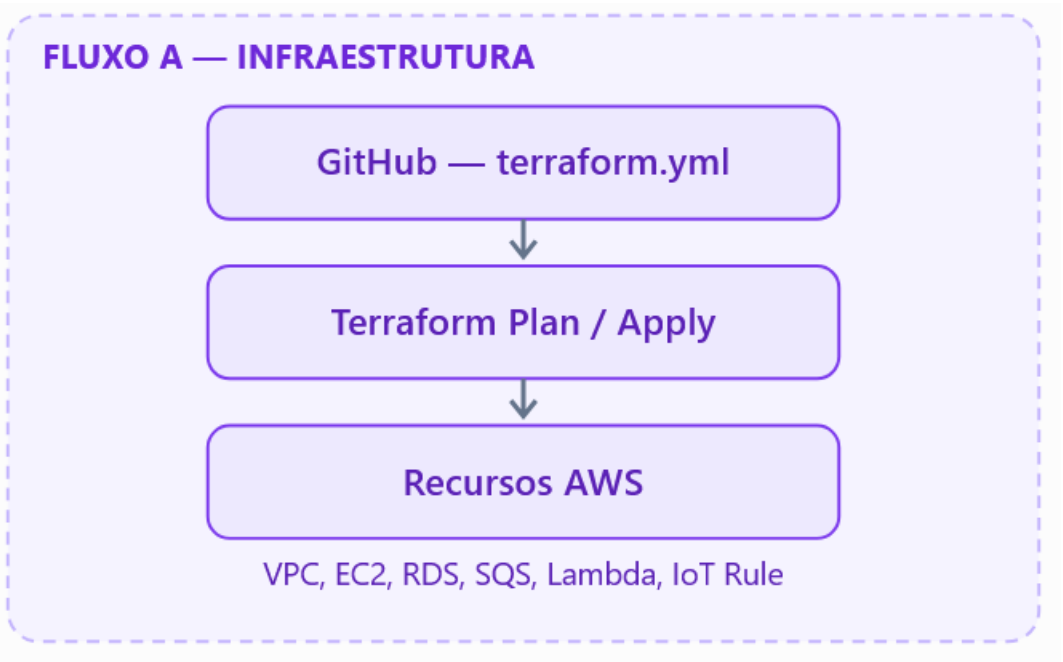


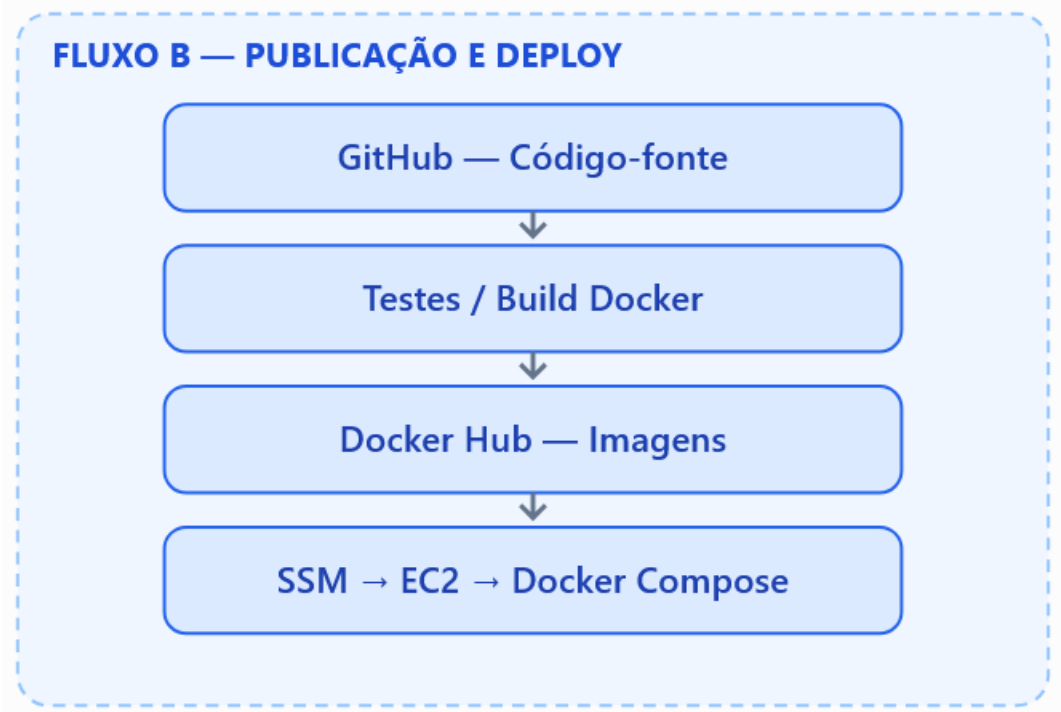
11. AWS Systems Manager — Deploy sem SSH

- O fluxo é:
  - O GitHub Actions inicia o workflow de deploy.
  - O workflow utiliza a API HTTPS da AWS para enviar um comando ao Systems Manager.
  - O Systems Manager encaminha o comando à instância gerenciada.
  - A EC2 executa as operações de atualização dos containers.
  - A instância baixa as imagens do Docker Hub e inicia a versão configurada.
- O mecanismo depende de permissões IAM, agente SSM funcional e conectividade com os endpoints necessários.
- Importante: SSM não significa que a EC2 dispensa qualquer conectividade de saída. Ela ainda precisa alcançar os serviços utilizados na administração e no download das imagens.



12. Arquitetura completa de CI/CD e provisionamento





12. Arquitetura completa de CI/CD e provisionamento

- Qual é a diferença prática?
  - Quando precisamos criar uma nova fila SQS, alterar um Security Group ou provisionar um banco, utilizamos o fluxo de Terraform.
  - Quando alteramos o código do React, Java ou Flask, utilizamos o fluxo de publicação e deploy dos containers.
  - Essa separação evita reprovisionar desnecessariamente toda a infraestrutura a cada atualização da aplicação.

Factory Hub

Arquitetura Completa da Solução

Da coleta de dados na planta à inteligência preditiva na nuvem



1 Automação Industrial



ESP32 / Wokwi
(Sensores de produção)

MQTTS

2 Ingestão AWS

AWS IoT Core → IoT Rule (Filtro e roteamento) → Amazon SQS (Fila de mensagens) → AWS Lambda (Processa e persiste dados)

Falhas após tentativas

DLQ – Dead Letter Queue
(Mensagens com falha)

3 Infraestrutura AWS

Amazon EC2 — Docker Compose

docker

React + Nginx
(Frontend)
Dashboards
Telemetria • OEE • Alertas

Java / Spring Boot
(Backend)
APIs REST
Regras de negócio
OEE • Consolidação

Python / Flask
(IA / ML)
Treinamento
Predições
Random Forest

Amazon RDS PostgreSQL
(Dados consolidados)
- Produção
- OEE
- Alertas
- Cadastros

Grava telemetria

InfluxDB
(Séries temporais)
Dados brutos (telemetria)

Modelo Random Forest
(Detecção de falhas e predições)

Usuário / Navegador
(Gestão e monitoramento)

HTTPS

Provisiona
(Ingestão: IoT, SQS, Lambda)

4 DevOps e IaC (Integração Contínua, Deploy e Provisionamento)

GitHub
(Código-fonte)

GitHub Actions
(CI/CD)
Build • Testes • Push • Deploy

Docker Hub
(Repositório de imagens)
Build / Push de imagens

Pull de imagens

AWS SSM Run Command
(Deploy na EC2)
Atualiza containers (Docker Compose)

Atualiza containers (Docker Compose)

Terraform
(IaC – Infraestrutura)
Provisiona recursos AWS

Amazon S3
(Terraform State)
Estado da infraestrutura (backend remoto)

Provisiona
(Infraestrutura: EC2, RDS, SG, Redes)

---

1 Automação Industrial
Coleta de dados da planta com ESP32 (ou Wokwi) e envio via MQTTS.

2 Ingestão AWS
Recebe, filtra e processa as mensagens com IoT Core, IoT Rule, SQS, DLQ e Lambda.

3 Infraestrutura AWS
Execução dos serviços em EC2 (Docker), armazenamento de dados no InfluxDB (brutos) e no RDS PostgreSQL (consolidados) e modelo de IA (Random Forest).

4 DevOps e IaC
CI/CD com GitHub Actions, Docker Hub e deploy na EC2 via SSM, e provisionamento da infraestrutura com Terraform (estado no S3).

Principais Resultados do Factory Hub

- Telemetria em tempo real
- Cálculo de OEE
- Alertas operacionais
- IA com Random Forest (predição de falhas)
- Infraestrutura na AWS
- CI/CD e Infraestrutura como Código (Terraform)

13. Pontos críticos da implantação

Aspecto	Risco ou verificação
AWS Academy	Expiração de credenciais e restrições IAM
EC2 única	Ponto único de falha para os containers
InfluxDB	Persistência dos volumes e backups
RDS	Conectividade privada, credenciais e backups
SQS/Lambda	Duplicidade, retries e DLQ
Docker	Versionamento, health checks e rollback
SSM	Permissões, conectividade e execução de comandos
Terraform	Proteção do state e controle de mudanças
GitHub Actions	Secrets, permissões e quality gates
Machine Learning	Persistência e versionamento dos modelos